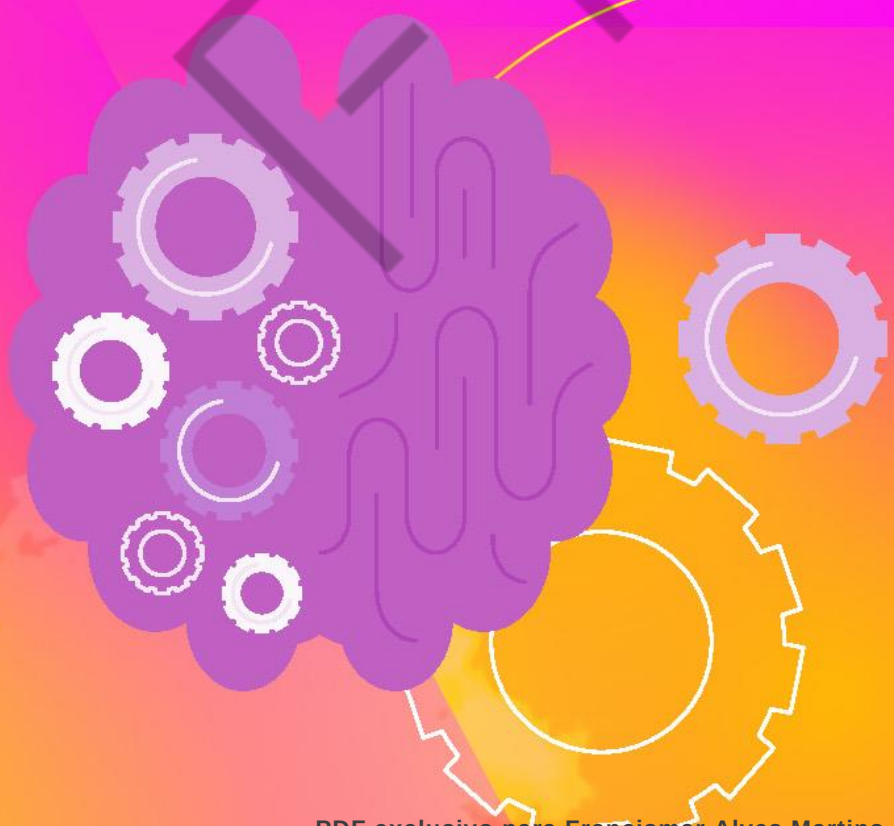
FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

EXPLORANDO O PODER DO APRENDIZADO POR REFORÇO



11

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - O que é um modelo avançado de Machine Learning?.....6
Figura 2 - Elementos da Engenharia de Features.....8
Figura 3 - Exemplos de features que podem ser removidas 10
Figura 4 - Exemplo de dataset desbalanceado 12
Figura 5 - Artigo sobre alternativas ao tratamento de dados desbalanceados.....15
Figura 6 - Ilustração de modelo ensemble sobre modelos simples.....17
Figura 7 - Explicação sobre respostas enviesadas (*bias*) ou variantes (*variance*)....18
Figura 8 - Intuição do método de *ensemble* por *bagging* 19
Figura 9 - Intuição do método de *ensemble* por *boosting* 21
Figura 10 - Intuição do método de *ensemble* por *voting* 23
Figura 11 - Intuição do método de *ensemble* por *stacking*.....25
Figura 12 - Relação inversa entre performance e interpretabilidade de modelos 27
Figura 13 - Exemplo de saída do método ELI5 sobre o dataset de tipos de vinho....29
Figura 14 - Exemplo de saída do método LIME sobre o dataset de qualidade de vinhos.....30
Figura 15 - Exemplo de saída do método SHAP sobre o dataset de predição de mortalidade dentro de 21 dias em decorrência de COVID-19.....31
Figura 16 - PyCaret, uma das principais bibliotecas para gerar modelos de Machine Learning de forma simplificada com pouca programação 34
Figura 17 - Saída do código-fonte 1 38
Figura 18 - Saída parcial do código-fonte 2.....39
Figura 19 - Saída do código-fonte 3 40
Figura 20 - Saída do código-fonte 4 41
Figura 21 - Saída do código-fonte 5 41
Figura 22 - Saídas do código-fonte 6 42
Figura 23 - Saídas do código-fonte 7 43
Figura 24 - Analogia do aprendizado por reforço 44
Figura 25 - Componentes do aprendizado por reforço.....45
Figura 26 - Técnicas intercaláveis necessárias ao aprendizado por reforço.....47
Figura 27 - Saída do código-fonte 10 53
Figura 28 - Saída do código-fonte 11 55
Figura 29 - Saída do código-fonte 12 56
Figura 30 - Saída do código-fonte 14 57
Figura 31 - Saída do código-fonte 15 58
Figura 32 - Saída do código-fonte 17 60
Figura 33 - Saída do código-fonte 18 62
Figura 34 - Saída do código-fonte 19 63
Figura 35 - Saída do código-fonte 20 65
Figura 36 - Saída do código-fonte 21 66

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Vantagens e desvantagens do PyCaret ou outras ferramentas low code 35

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Instalação da biblioteca, imports e leitura do dataset 37
Código-fonte 2 - Divisão de TREINO e TESTE e setup do experimento com PyCaret 38
Código-fonte 3 - Comparando algoritmos de regressão e ordenando pelo R^2 39
Código-fonte 4 - Criando um único modelo baseado em Gradient Boosting.....40
Código-fonte 5 - Tunando o modelo baseado em Gradient Boosting.....41
Código-fonte 6 - Plotando detalhes do modelo de Gradient Boosting tunado 42
Código-fonte 7 - Predizendo os exemplos do teste.....42
Código-fonte 8 - Finalizando o modelo (retreino com o dataset inteiro) 43
Código-fonte 9 - Imports e funções de apoio 52
Código-fonte 10 - Leitura dos dados e Análise exploratória.....53
Código-fonte 11 - Identificação de Outliers 54
Código-fonte 12 - Plotando a matriz de correlação 55
Código-fonte 13 - Tratamento dos dados.....56
Código-fonte 14 - Plotando a distribuição das classes.....57
Código-fonte 15 - Atenuando o desbalanceamento das classes.....58
Código-fonte 16 - Engenharia de Features 59
Código-fonte 17 - Gerando o modelo baseline.....59
Código-fonte 18 - Gerando modelos por diferentes estratégias de ensemble 62
Código-fonte 19 - Plotando matrizes de confusão.....62
Código-fonte 20 - Explicando um exemplo de teste via LIME 64
Código-fonte 21 - Explicabilidade geral do modelo via SHAP Values 65

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 EXPLORANDO O PODER DO APRENDIZADO POR REFORÇO 6
1.1 Introdução 6
2 PREPARAÇÃO DE DADOS AVANÇADA 8
2.1 Tratamento de features 8
2.1.1 Uma visão negocial 8
2.1.2 Uma visão técnica 10
2.2 Desbalanceamento 12
2.2.1 Definição e discussão.....12
2.2.2 Técnicas para lidar com desbalanceamento 14
2.2.3 Uma reflexão sobre desbalanceamento 15
3 MÉTODOS DE ENSEMBLE.....17
3.1 Motivação.....17
3.2 Bagging 19
3.3 Boosting 20
3.4 Voting 22
3.5 Stacking.....25
4 INTERPRETABILIDADE DE MODELOS.....27
4.1 Qual a importância? 27
4.2 ELI5.....28
4.3 LIME.....29
4.4 SHAP.....31
5 SIMPLIFICANDO A CRIAÇÃO DE MODELOS PREDITIVOS 34
5.1 Facilitando o acesso à ciência de dados.....34
5.2 Vantagens e Desvantagens 34
5.3 Introdução ao PyCaret.....35
5.3.1 Usos gerais e como se aprofundar na ferramenta 35
5.3.2 Hands On: Modelagem Preditiva via PyCaret 36
6 APRENDIZADO POR REFORÇO.....44
6.1 Introdução 44
6.2 Componentes e Aplicações.....45
6.3 Funcionamento.....46
7 HANDS ON 50
7.1 Sobre a base de dados 50
7.2 Preparação da base 51
7.3 Ensembles.....60
7.4 Interpretabilidade.....64
CONCLUSÃO.....67
REFERÊNCIAS.....68
GLOSSÁRIO 70

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 EXPLORANDO O PODER DO APRENDIZADO POR REFORÇO

1.1 Introdução

Os métodos avançados de Machine Learning (ML) são técnicas mais sofisticadas e robustas, que foram projetadas para lidar com desafios complexos que vão além das abordagens tradicionais. À medida que o campo evolui, surge a necessidade de métodos que sejam capazes de tratar não apenas a modelagem preditiva, mas também problemas que afetam a **qualidade dos dados**, a capacidade de **explicação dos modelos** e o **desempenho em cenários reais**. Esses métodos oferecem soluções para dificuldades enfrentadas em projetos mais complexos e proporcionam insights mais profundos e precisos.

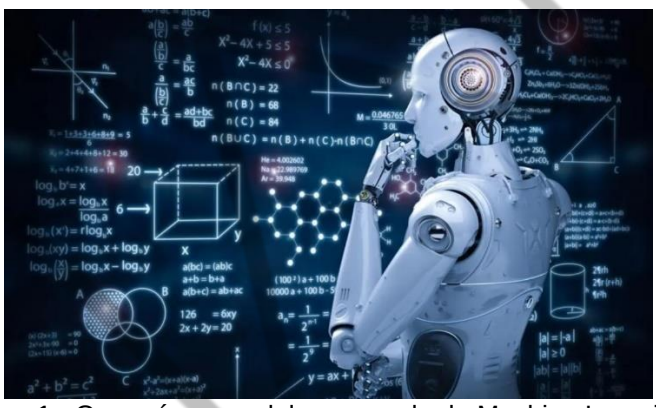


Figura 1 - O que é um modelo avançado de Machine Learning?
Fonte: LinkedIn (2024)

Quando falamos de “avançado”, nos referimos a diferentes aspectos do pipeline de Machine Learning. Primeiramente, a **preparação avançada de dados** envolve lidar com dados mais desafiadores, como desbalanceamento de classes, dados ruidosos e variáveis de diferentes naturezas. Saber como preparar corretamente os dados é fundamental para garantir que os modelos possam extrair informações valiosas, e métodos avançados como técnicas de amostragem, manipulação de variáveis e criação de features desempenham um papel crucial.

Além disso, a **modelagem avançada** inclui o uso de algoritmos mais sofisticados, como métodos de ensemble, que combinam múltiplos modelos para melhorar a performance. Esses métodos permitem aumentar a precisão e a robustez

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

das previsões, lidando melhor com problemas como overfitting, variabilidade e generalização em conjuntos de dados complexos. Aprender essas técnicas eleva sua capacidade de construir modelos de alta qualidade, adequando em aplicações mais exigentes.

É importante mencionar que a **interpretação avançada de resultados** vai além das métricas tradicionais de acurácia ou erro. Em problemas reais, é essencial entender por qual motivo e como um determinado modelo tomou determinadas decisões. Ferramentas de interpretabilidade ajudam a explicar os resultados, fornecendo insights críticos sobre o comportamento dos modelos. Compreender essa interpretação é vital para construir modelos mais transparentes e confiáveis, especialmente em áreas sensíveis como saúde, finanças e decisões automatizadas que afetam pessoas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 PREPARAÇÃO DE DADOS AVANÇADA

2.1 Tratamento de features

O tratamento de features em qualquer modelo preditivo deve equilibrar aspectos técnicos e negociais, pois ambos desempenham papéis essenciais na construção de um modelo eficaz (figura “Elementos da Engenharia de Features”).

Figura 2 - Elementos da Engenharia de Features
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

Do ponto de vista **técnico**, técnicas como análise de correlação, seleção de variáveis e padronização de features ajudam a garantir que o modelo seja eficiente e preciso, evitando redundâncias e priorizando variáveis relevantes. No entanto, essas decisões técnicas devem sempre ser orientadas pelo conhecimento do **negócio**, que oferece o contexto necessário para entender a importância real de cada feature.

Um dado tecnicamente relevante pode não ter impacto prático no problema, enquanto uma variável aparentemente secundária pode ser crucial para a tomada de decisões estratégicas. Assim, alinhar a preparação das features com os objetivos de negócio é fundamental para garantir que o modelo final seja não apenas preciso, mas também útil e aplicável ao contexto real.

2.1.1 Uma visão negocial

Antes de entrarmos com a limpeza de dados ou engenharia de features clássica (como tratamento de *missings* ou padronização de dados), podemos pensar no negócio como um direcionador fundamental das ações que tomamos sobre os dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Cada conjunto de dados reflete uma realidade específica e, para extrair o valor real, é crucial refletir sobre como cada feature (variável) se relaciona com o problema que estamos tentando resolver. Por exemplo, em um projeto para prever *churn* de clientes, dados como o histórico de uso, o tempo de relacionamento com a empresa e as interações de suporte são cruciais para o entendimento do comportamento do cliente. Nesse sentido, **o conhecimento do domínio pode ajudar a identificar quais variáveis podem ser mais relevantes e quais podem ser descartadas ou tratadas de maneira diferenciada**.

A relevância de cada feature em relação às demais deve ser analisada com cuidado, especialmente porque muitas vezes algumas variáveis carregam mais informações do que outras, e nem sempre isso é imediatamente aparente. Técnicas como análise de correlação ou *feature importance* (importância das variáveis) em árvores de decisão podem ajudar a entender o peso de cada variável no modelo, mas, novamente, é o conhecimento do negócio que orienta quais correlações ou pesos fazem sentido no contexto. Nem sempre uma alta correlação implica em importância real para o problema; às vezes, uma variável aparentemente secundária pode ser essencial para representar um fator específico do comportamento dos usuários, algo que somente quem compreende o negócio profundamente pode perceber.

Além disso, podemos utilizar técnicas de *feature selection* (seleção de variáveis) e *feature engineering* com o foco no impacto prático dos dados. A eliminação de variáveis redundantes ou irrelevantes pode simplificar o modelo e melhorar seu desempenho, mas é importante sempre perguntar: "**Como essa feature se conecta com o nosso objetivo de negócio?**". A criação de novas features, por outro lado, pode capturar relações não lineares ou oferecer novas perspectivas sobre os dados, como combinações de variáveis ou agregações temporais que refletem comportamentos relevantes.

Portanto, ao alinhar a manipulação de features com o entendimento profundo do negócio, conseguimos não apenas otimizar o desempenho técnico do modelo, mas também garantir que ele esteja diretamente vinculado aos objetivos estratégicos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.1.2 Uma visão técnica

O tratamento de features em modelos preditivos envolve uma série de decisões técnicas que ajudam a definir quais variáveis são mantidas e quais podem ser descartadas ou transformadas. A **redução de dimensionalidade** é uma dessas estratégias, que visa simplificar o conjunto de dados ao manter apenas as features mais relevantes, sem sacrificar a capacidade preditiva do modelo. O processo de identificar e remover features pouco informativas é essencial para melhorar a eficiência do modelo, reduzir o risco de overfitting e tornar os resultados mais interpretáveis.

Feature A	Feature B	Feature C	Feature D	Feature E	Feature F	Feature G
5,25	nível 1	23	69	NaN	100	verde
9,34	nível 1	25	75	-9	111	verde
2,23	nível 1	9	27	NaN	115	amarelo
5,84	nível 1	11	33	NaN	56	azul
6,78	nível 1	25	75	NaN	100	vermelho
10,85	nível 1	13	39	-3	25	verde
92,50	nível 1	13	39	NaN	83	azul
87,20	nível 1	19	57	NaN	95	vermelho
69,11	nível 1	21	63	NaN	87	amarelo

Figura 3 - Exemplos de features que podem ser removidas
Fonte: Elaborado pelo autor (2024)

Alguns indicadores técnicos fornecem pistas valiosas sobre quais variáveis podem ser descartadas ou transformadas, e cada uma dessas pistas é baseada em como as features se comportam dentro do conjunto de dados:

- **Muitos valores nulos:** features com muitos valores ausentes (*missing values*) geralmente trazem pouca informação ou são difíceis de tratar adequadamente. **Uma regra prática é considerar descartar variáveis que tenham uma proporção significativa de valores nulos, especialmente se esses valores não puderem ser imputados de maneira confiável**. Este é o caso da *Feature E* da figura “Exemplos de features que podem ser removidas”.
- **Baixa variância:** variáveis com pouca variação, ou seja, aquelas em que a maioria dos dados assume o mesmo valor ou valores muito próximos, tendem a ser pouco informativas. Um exemplo clássico seria uma variável

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

binária onde 99% dos dados têm o valor "0" e apenas 1% têm "1". Nesse caso, **a baixa variância indica que essa feature não contribuirá significativamente para o modelo preditivo**. O descarte de variáveis com baixa variância pode simplificar o modelo sem perda de performance. Outro exemplo é a *Feature B* da figura “Exemplos de features que podem ser removidas”.

- **Métodos baseados em árvores:** algoritmos como Random Forests ou XGBoost são naturalmente eficientes em lidar com features irrelevantes ou pouco informativas, já que eles selecionam automaticamente as variáveis mais importantes durante o processo de criação das árvores. **Ao analisar a importância das features derivada de modelos de árvores, é possível identificar quais variáveis têm pouco impacto nas decisões do modelo**. Variáveis com importância próxima de zero podem ser candidatas ao descarte sem prejudicar o desempenho do modelo.
- **Alta correlação entre si:** features altamente correlacionadas (multicolinearidade) indicam que algumas variáveis estão essencialmente representando a mesma informação, o que pode gerar redundância e confundir o modelo. Nesse caso, **é recomendável descartar uma das features correlacionadas, mantendo apenas a mais relevante**. Por exemplo, se em um modelo de previsão de preço de imóveis as variáveis “número de quartos” e “área total” tiverem alta correlação, pode ser mais útil manter apenas uma delas para evitar redundância e simplificar o modelo. As *Features C* e *D* da figura “Exemplos de features que podem ser removidas” expressam a mesma informação (D = 3*C), então uma delas pode ser suprimida.
- **Análise de Componentes Principais (PCA):** essa técnica pode ser utilizada para identificar as features que melhor representam a variabilidade dos dados. Features que não contribuem significativamente para a explicação da variância global podem ser descartadas sem impacto relevante no desempenho do modelo. O PCA, **ao combinar várias variáveis em componentes principais, revela quais variáveis originais podem ser eliminadas**, pois contribuem muito pouco para o comportamento geral dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esses aspectos técnicos fornecem uma base sólida para a seleção e eliminação de features durante a preparação dos dados. O objetivo é simplificar o modelo, melhorar sua capacidade de generalização e reduzir o tempo de processamento.

Ao remover variáveis redundantes, pouco informativas ou problemáticas, o modelo se torna mais robusto e eficiente. Essas decisões, quando bem fundamentadas em análise técnica, também ajudam a evitar o overfitting e a melhorar a interpretabilidade dos resultados, especialmente em projetos de Machine Learning com grandes volumes de dados e muitas variáveis.

2.2 Desbalanceamento

2.2.1 Definição e discussão

O desbalanceamento de dados ocorre quando as classes em um conjunto de dados não estão representadas de forma equilibrada, ou seja, uma classe é muito mais frequente do que outra. Isso é especialmente comum em problemas de **classificação**, onde a proporção entre as classes pode afetar diretamente o desempenho do modelo (figura “Exemplo de dataset desbalanceado”).

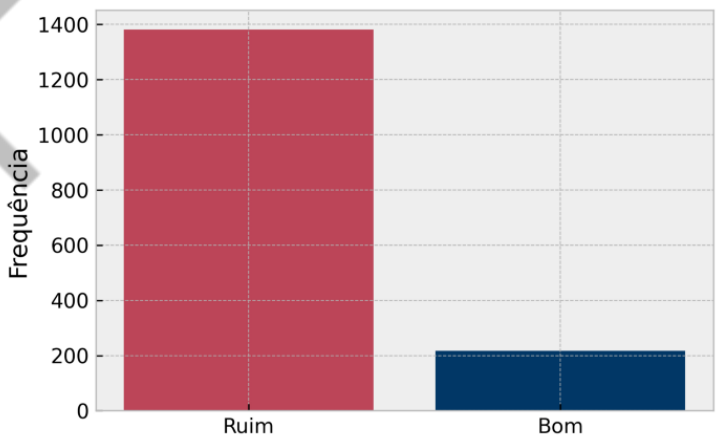


Figura 4 - Exemplo de dataset desbalanceado
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por exemplo, em um problema de detecção de fraudes, as transações fraudulentas geralmente representam uma pequena fração do total de transações. Se o modelo for treinado em um conjunto de dados altamente desbalanceado, ele pode aprender a simplesmente prever a classe majoritária (não fraude) e ainda assim obter alta acurácia, mascarando a incapacidade de identificar casos minoritários (fraude).

O desbalanceamento de dados é menos problemático em situações de regressão, pois nesses casos, o objetivo é prever um valor contínuo em vez de uma classe categórica. No entanto, é na classificação que o desbalanceamento se torna crítico, pois pode influenciar a maneira como o modelo aprende e faz previsões. Modelos treinados em conjuntos de dados desbalanceados tendem a ter um viés para a classe majoritária, resultando em métricas enganosamente elevadas, como alta acurácia, mas baixa sensibilidade e especificidade.

Apesar disso, é importante reconhecer que o desbalanceamento de dados não é necessariamente um problema em todas as análises. Em análises descritivas, o desbalanceamento é uma característica intrínseca dos dados, refletindo a distribuição real de um fenômeno. Por exemplo, se uma empresa está analisando o perfil de clientes que compram um produto premium, a desproporção entre os clientes que compram e os que não compram pode ser um reflexo real do mercado. Nesse contexto, a análise busca descrever o comportamento das classes e não fazer previsões, portanto, a representação desigual é apenas um aspecto da realidade dos dados e não um problema a ser resolvido.

Outro ponto a considerar é que nem todas as classes desbalanceadas são problemáticas. Em alguns casos, classes muito distintas e sem sobreposição significativa podem ser bem separadas por modelos, mesmo em um cenário desbalanceado. Por exemplo, em um problema de detecção de doenças raras onde as características dos doentes e dos saudáveis são suficientemente distintas, o desbalanceamento não necessariamente compromete a precisão do modelo. No entanto, é fundamental avaliar a natureza do problema, a complexidade do modelo e as métricas de desempenho para determinar se o desbalanceamento deve ser tratado ou se pode ser ignorado como uma característica dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2.2 Técnicas para lidar com desbalanceamento

Lidar com dados desbalanceados é uma etapa crucial no desenvolvimento de modelos de Machine Learning para evitar que eles se tornem enviesados em favor da classe majoritária. Existem várias abordagens que podem ser utilizadas para tratar esse desbalanceamento, desde soluções simples até técnicas mais complexas que alteram a estrutura do conjunto de dados. Uma das estratégias iniciais é **coletar mais dados**, o que pode ajudar a aumentar a representatividade da classe minoritária. Embora nem sempre seja viável devido às limitações de tempo, orçamento ou disponibilidade, essa abordagem pode melhorar significativamente a distribuição do dataset.

Outra forma de lidar com o desbalanceamento é **mudar a estrutura do dataset**, que pode ser feita de duas maneiras principais: undersampling e oversampling.

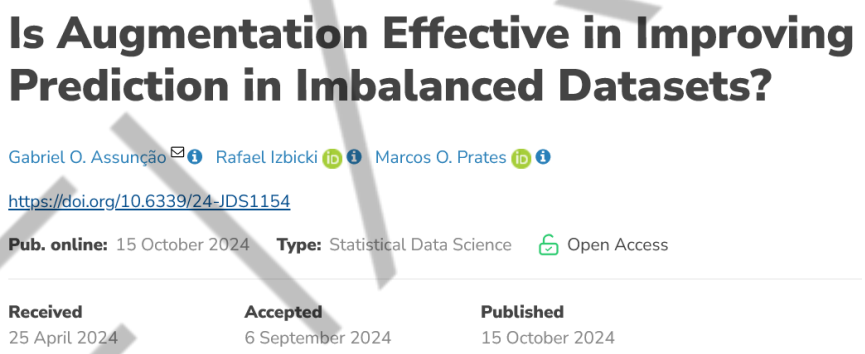
- No **undersampling**, reduzimos o número de instâncias da classe majoritária para equilibrar as classes. Uma técnica comum é o Random Undersampling, que remove aleatoriamente amostras da classe majoritária. No entanto, isso pode levar à perda de informações importantes, reduzindo a variabilidade dos dados e tendo o potencial de impactar negativamente a performance do modelo. Uma variação desse método é “unir” duas ou mais instâncias próximas da classe majoritária para criar uma representação mais compacta, mantendo alguma variabilidade sem perder muita informação.
- No **oversampling,** buscamos aumentar o número de exemplos da classe minoritária. Uma técnica bem conhecida para fazer isso é o SMOTE (Synthetic Minority Over-sampling Technique), que cria exemplos sintéticos da classe minoritária em vez de apenas replicar amostras existentes. O SMOTE gera novos exemplos interpolando entre os pontos existentes da classe minoritária, criando uma distribuição mais realista e reduzindo o risco de overfitting que ocorre com a simples replicação dos dados. Essa abordagem ajuda a melhorar a representatividade da classe minoritária e pode levar a modelos mais robustos e precisos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Essas técnicas de modificação da estrutura do dataset podem ser escolhidas e ajustadas de acordo com o problema em questão e a natureza dos dados. O objetivo é alcançar um equilíbrio entre as classes que permita ao modelo aprender de forma mais eficaz, aumentando a precisão e a confiabilidade das previsões em cenários de dados desbalanceados.

2.2.3 Uma reflexão sobre desbalanceamento

A técnica de aumento de dados pode ser eficaz para melhorar as previsões em conjuntos de dados desbalanceados, mas sua efetividade depende de como é aplicada e do contexto do problema. Pesquisas recentes como, por exemplo, a apresentada na figura “Artigo sobre alternativas ao tratamento de dados desbalanceados”, demonstram que o aumento de dados não é a única resposta para dados desbalanceados.


Is Augmentation Effective in Improving Prediction in Imbalanced Datasets?

Gabriel O. Assunção Rafael Izbicki Marcos O. Prates

https://doi.org/10.6339/24-JDS1154

Pub. online: 15 October 2024 Type: Statistical Data Science Open Access

Received 25 April 2024 Accepted 6 September 2024 Published 15 October 2024

Figura 5 - Artigo sobre alternativas ao tratamento de dados desbalanceados
Fonte: Elaborado pelo autor (2024)

Em muitos casos, ao lidar com desbalanceamento de dados, manipular diretamente a estrutura do dataset pode não ser a abordagem ideal, pois isso altera a distribuição natural das classes e pode introduzir vieses indesejados no modelo. Para respeitar melhor a integridade dos dados originais, uma estratégia menos intrusiva é **calibrar o modelo e ajustar o ponto de corte (*threshold*) de decisão**. Alterando o *threshold*, é possível ajustar o ponto em que o modelo classifica uma instância em uma classe ou outra, sem modificar diretamente os dados. Por exemplo, em um problema de detecção de fraude, onde a classe “fraude” é muito menor, podemos ajustar o *threshold* para tornar o modelo mais sensível a essa classe, aumentando a probabilidade de detectar fraudes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Outra abordagem para lidar com o desbalanceamento sem mexer nos dados é **escolher algoritmos mais resistentes ao desbalanceamento**, como o Gradient Boosting. Esse tipo de algoritmo utiliza uma série de modelos fracos, como árvores de decisão, para criar um modelo forte e ajusta automaticamente o peso das classes ao longo do processo de aprendizado. Por isso, ele é mais capaz de lidar com desbalanceamento ao dar maior peso às instâncias da classe minoritária. Dessa forma, o modelo aprende a identificar padrões na classe menor sem precisar de modificações nos dados.

Além disso, é possível **utilizar modelos penalizados**, que aplicam diferentes funções de custo para diferentes classes. Em um modelo penalizado, o erro de classificação de uma instância da classe minoritária pode receber uma penalidade maior, incentivando o modelo a dar mais atenção a essa classe. Essa técnica é especialmente útil em algoritmos de regressão logística penalizada, onde o erro para a classe menor é amplificado pela função de custo, ajudando o modelo a equilibrar a importância entre as classes sem alterar o dataset.

Para finalizar, outra abordagem fundamental para lidar com dados desbalanceados é **usar as métricas corretas de avaliação e outras métricas de treino mais adequadas ao problema**. Métricas como a acurácia podem ser enganosas em cenários desbalanceados, pois o modelo pode acertar a maioria das previsões simplesmente favorecendo a classe majoritária. Em vez disso, é recomendável utilizar métricas como F1-Score, AUC-ROC e Precision-Recall, que dão uma visão mais precisa do desempenho do modelo em cada classe, especialmente nas classes minoritárias. Ao treinar e avaliar o modelo com essas métricas, conseguimos uma visão mais equilibrada e realista do desempenho, garantindo que o modelo seja efetivo em cenários desbalanceados sem comprometer a estrutura natural dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 MÉTODOS DE ENSEMBLE

3.1 Motivação

Ensemble learning é uma abordagem em Machine Learning que busca combinar múltiplos modelos para criar um sistema mais robusto e eficaz, capaz de aprender a partir dos dados de maneira mais abrangente. A ideia por trás dos métodos de ensemble é simples: em vez de depender de um único modelo para fazer previsões, vários modelos são treinados e seus resultados são combinados de forma a produzir uma resposta final que considera a contribuição de cada um. Isso melhora a capacidade de generalização, uma vez que o ensemble consegue captar padrões que modelos individuais podem não detectar (figura “Ilustração de modelo ensemble sobre modelos simples”).

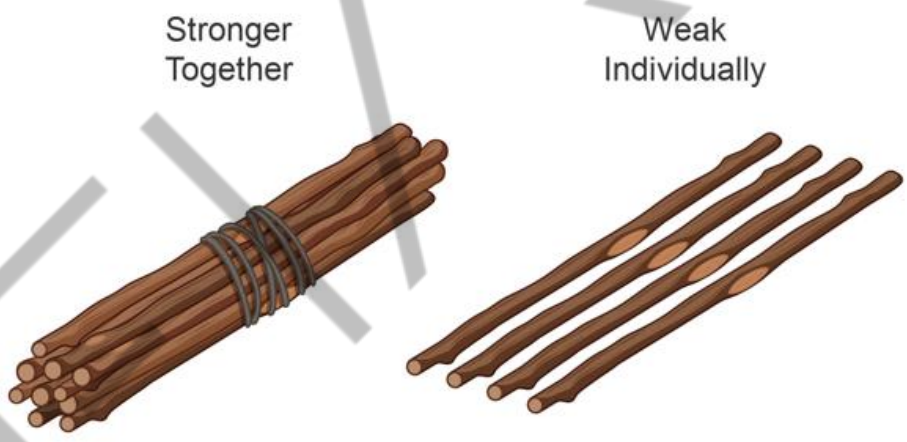


Figura 6 - Ilustração de modelo ensemble sobre modelos simples
Fonte: Daily Dose of Data Science (2023), adaptado por FIAP (2024)

Cada algoritmo de Machine Learning possui seus prós e contras, ou seja, o desempenho pode variar de acordo com as características do problema. Um modelo específico pode ser hiper-ajustado e encontrar certos padrões nos dados, mas também pode falhar em generalizar, captando apenas aspectos específicos que não representam bem novos dados. Da mesma forma, diferentes algoritmos podem ter suas próprias limitações, e é impossível garantir que um único método seja ótimo para todos os problemas. Por isso, contar com um único modelo pode limitar a eficácia das previsões e aumentar o risco de overfitting ou underfitting.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A grande vantagem do ensemble learning é que ele permite explorar a diversidade de modelos e maximizar suas forças individuais. Cada modelo em um ensemble contribui com sua própria interpretação dos padrões nos dados, o que resulta em uma resposta final mais equilibrada e precisa. Em outras palavras, a técnica de ensemble permite gerar modelos com menor viés e menor variância das respostas. Observe a figura “Explicação sobre respostas enviesadas (*bias*) ou variantes (*variance*)”.

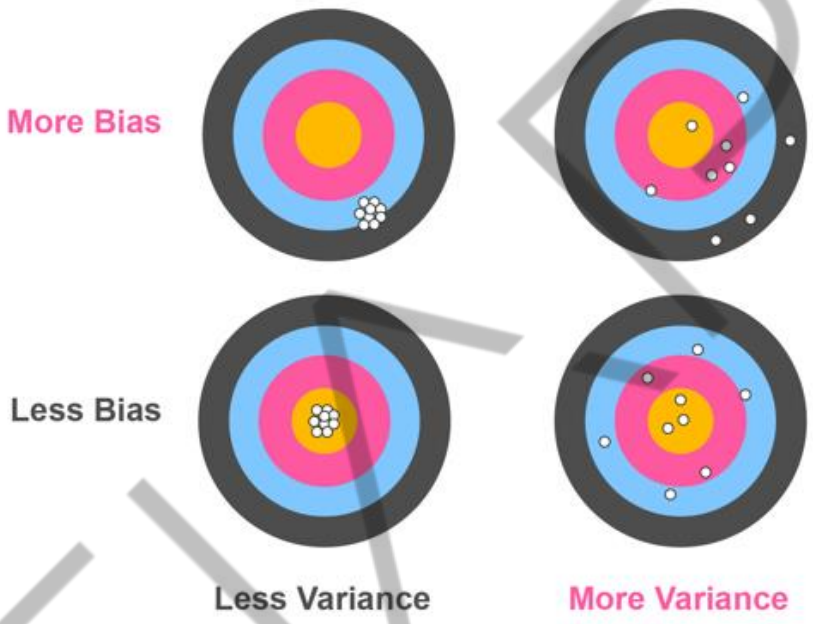


Figura 7 - Explicação sobre respostas enviesadas (*bias*) ou variantes (*variance*)
Fonte: Medium (2019), adaptado por FIAP (2024)

Modelos de ensemble podem gerar modelos preditivos com menos viés e menor variância ao combinar as forças de múltiplos modelos individuais. Ao unir previsões de diferentes algoritmos, eles equilibram os erros, reduzindo a tendência de um único modelo cometer os mesmos tipos de erros (viés) e evitando que resultados muito distintos ocorram devido às variações nos dados de treinamento (variância).

Existem diferentes técnicas de ensemble. Conheceremos as principais a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2 Bagging

Bagging (Bootstrap Aggregating) é uma técnica de ensemble que melhora a estabilidade e precisão de modelos de Machine Learning, reduzindo a variância e ajudando a evitar o overfitting. A ideia central do bagging é treinar vários modelos independentes em subconjuntos aleatórios de dados e combinar suas previsões. Esses subconjuntos são criados utilizando a técnica de bootstrap, onde amostras do conjunto de dados original são retiradas com reposição, permitindo que a mesma instância possa aparecer em diferentes subconjuntos, como ilustrado na figura "Intuição do método de *ensemble* por *bagging*".

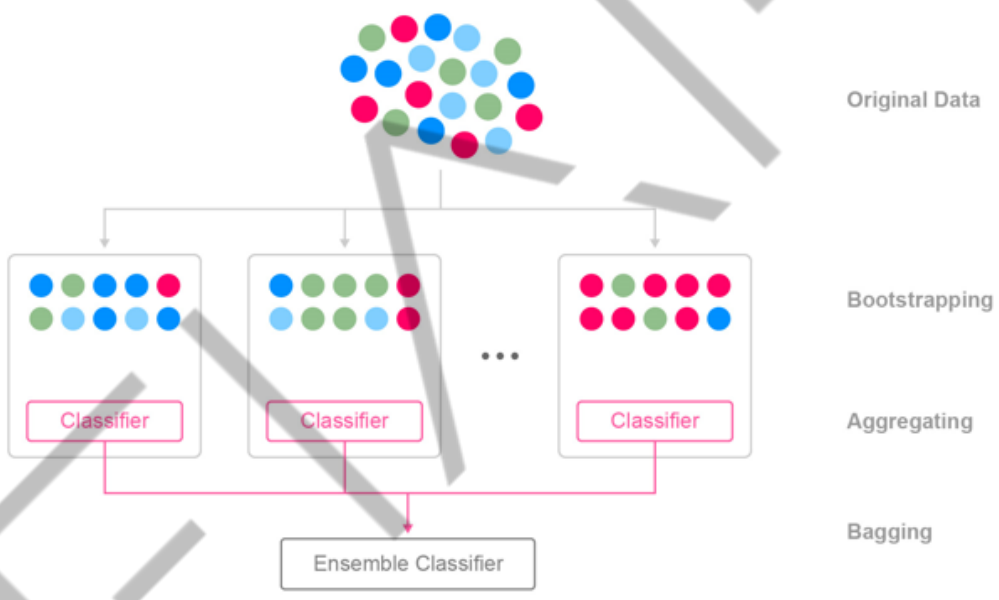


Figura 8 - Intuição do método de *ensemble* por *bagging*
Fonte: Medium (2023), adaptado por FIAP (2024)

Como funciona o Bagging:

1. O conjunto de dados original é dividido em vários subconjuntos por meio do bootstrap, criando múltiplas versões do dataset.
2. Um modelo é treinado em cada subconjunto, podendo ser o mesmo tipo de algoritmo ou diferentes tipos.
3. As previsões dos modelos são combinadas: em problemas de regressão, a média das previsões é usada; em problemas de classificação, a combinação é realizada por votação majoritária (o resultado mais frequente é escolhido).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplos de técnicas de Bagging:

- **Random Forest:** é um dos exemplos mais populares de bagging. Ele utiliza árvores de decisão como modelos base e cria múltiplas árvores em subconjuntos aleatórios dos dados. Além disso, ele introduz aleatoriedade na escolha das features em cada nó da árvore, o que aumenta a diversidade entre as árvores. A combinação das previsões das árvores (média para regressão ou votação para classificação) melhora a precisão e a generalização do modelo.
- **Bagging Classifier/Regressor:** implementado no Scikit-learn, o `BaggingClassifier` ou `BaggingRegressor` pode ser utilizado para aplicar o bagging a qualquer algoritmo de base. Por exemplo, pode-se criar um ensemble de modelos de regressão linear ou k-Nearest Neighbors (kNN), e as previsões finais serão a média ou votação dos resultados individuais.

Benefícios do Bagging:

O principal benefício do bagging é a redução da variância sem aumentar muito o viés. Ao treinar modelos em subconjuntos diferentes e combinar suas previsões, ele suaviza os erros que poderiam ocorrer em um único modelo que é sensível a variações nos dados. Isso é especialmente útil em modelos como árvores de decisão, que são naturalmente propensos ao overfitting quando treinados isoladamente.

3.3 Boosting

Boosting é uma técnica de ensemble que melhora o desempenho de modelos de Machine Learning ao combinar vários modelos fracos em um modelo forte. Ao contrário do bagging, onde os modelos são treinados de forma independente, **no boosting os modelos são treinados sequencialmente, e cada novo modelo é ajustado para corrigir os erros dos modelos anteriores**. Essa abordagem foca em aprender com os erros, criando um modelo mais preciso a cada iteração. O objetivo do boosting é reduzir o viés do modelo, ajustando-o cada vez mais para se aproximar dos resultados reais. Observe a figura “Intuição do método de *ensemble* por *boosting*”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

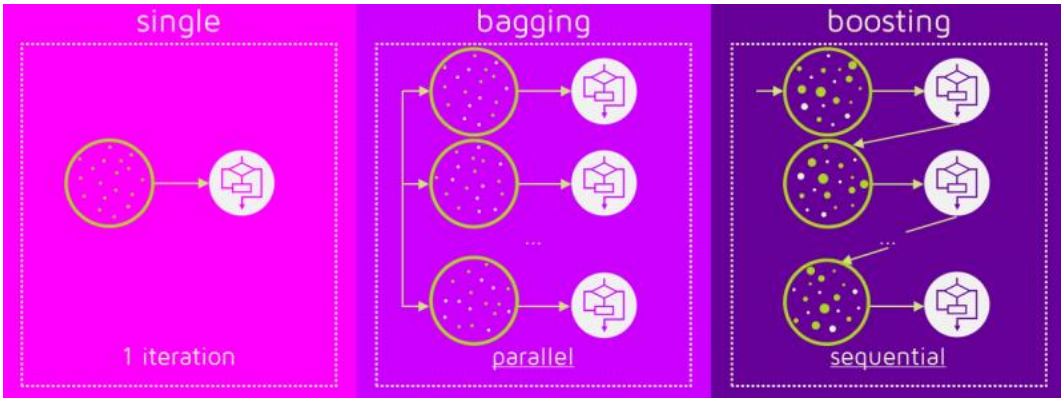


Figura 9 - Intuição do método de *ensemble* por *boosting*
Fonte: YouTube (2018)

Como funciona o Boosting:

1. Inicialmente, o primeiro modelo é treinado com pesos iguais para todas as observações.
2. Em seguida, os dados nos quais o primeiro modelo cometeu erros recebem maior peso ou atenção.
3. Um novo modelo é treinado, levando em conta essas observações pesadas, e suas previsões são combinadas com as do modelo anterior.
4. Esse processo continua, com cada novo modelo focando em corrigir os erros acumulados pelos anteriores.
5. No final, as previsões de todos os modelos são combinadas para formar a previsão final, frequentemente ponderada com base na precisão de cada modelo.

Exemplos de técnicas de Boosting:

- **AdaBoost (Adaptive Boosting):** no AdaBoost, o primeiro modelo é treinado e seus erros são identificados. Em seguida, o segundo modelo foca em corrigir esses erros dando mais peso às observações que foram mal classificadas pelo primeiro modelo. Esse processo continua até que um número pré-definido de modelos seja criado. No final, as previsões são combinadas, geralmente com uma votação ponderada para classificação ou uma média ponderada para regressão. AdaBoost é conhecido por melhorar

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

a precisão de modelos de classificação, especialmente em problemas onde os dados são lineares.

- **Gradient Boosting:** nessa técnica, cada novo modelo é treinado para minimizar o erro residual (a diferença entre as previsões e os valores reais) dos modelos anteriores. Isso é feito de forma incremental, onde cada novo modelo tenta reduzir o erro do modelo anterior. O Gradient Boosting pode ser aplicado a problemas de regressão e classificação, e é altamente eficaz em tarefas complexas.
- **XGBoost (Extreme Gradient Boosting):** uma versão otimizada do Gradient Boosting, o XGBoost inclui melhorias de velocidade e performance, como regularização adicional para reduzir o overfitting. Ele também é altamente eficiente em termos de memória e processamento, tornando-o popular para competições de Machine Learning e grandes volumes de dados. É amplamente utilizado em problemas de previsão e classificação onde a alta precisão é necessária.

Benefícios do Boosting:

O principal benefício do boosting é a sua capacidade de reduzir o viés e aumentar a precisão, especialmente em problemas complexos. Como cada modelo se concentra em corrigir os erros dos modelos anteriores, o boosting é particularmente eficaz em conjuntos de dados que possuem padrões difíceis de detectar. No entanto, é importante notar que o boosting pode ser propenso a overfitting se não for configurado corretamente, especialmente em datasets ruidosos.

3.4 Voting

Voting é uma técnica de ensemble que **combina as previsões de múltiplos modelos diferentes para gerar uma única previsão final**, de modo a aumentar a robustez e a precisão do modelo preditivo. Ao contrário do bagging e do boosting, em que geralmente se usa o mesmo tipo de modelo repetidamente (como múltiplas árvores de decisão), o **voting permite combinar diferentes algoritmos**, como uma combinação de modelos de árvore de decisão, regressão logística e k-NN, para tirar

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

proveito das forças de cada um. A técnica de voting é amplamente utilizada em problemas de classificação, mas também pode ser adaptada para regressão (figura "Intuição do método de *ensemble* por *voting*").

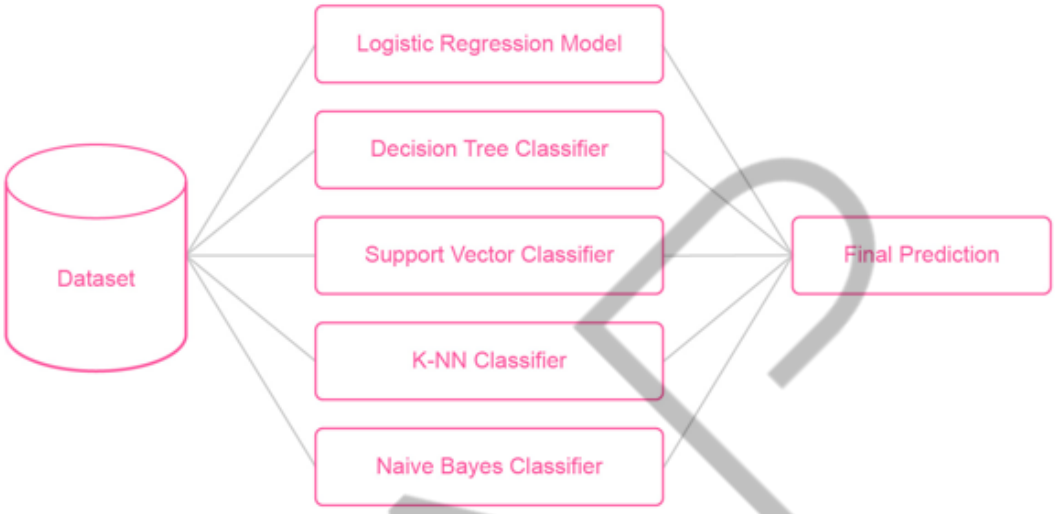


Figura 10 - Intuição do método de *ensemble* por *voting*
Fonte: Springer Nature Link (2023), adaptado por FIAP (2024)

Como funciona o Voting:

Existem duas principais abordagens para a combinação das previsões dos modelos em voting:

- **Voting Majoritário (Hard Voting):** nesse método, cada modelo gera uma previsão de classe para uma instância, e a classe que receber mais votos é a escolhida como previsão final. Por exemplo, se três modelos de classificação (árvore de decisão, k-NN e regressão logística) fazem previsões para uma determinada instância e dois deles preveem a classe "1" enquanto o terceiro prevê a classe "0", a previsão final será "1", pois essa classe teve a maioria dos votos.

- **Voting Ponderado (Soft Voting):** esse método leva em consideração as probabilidades preditivas de cada modelo. Cada modelo calcula a probabilidade de uma instância pertencer a cada classe, e a previsão final é feita com base na média dessas probabilidades ponderadas. Essa abordagem é útil quando alguns modelos são mais confiáveis que outros,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

permitindo que se atribuam pesos diferentes para cada modelo. Assim, modelos mais precisos podem ter maior influência na decisão final.

Exemplos de Voting:

- **Exemplo 1 — Diagnóstico médico:** em um problema de classificação para prever se uma doença está presente ou ausente, podemos combinar três modelos: um modelo de árvore de decisão, uma regressão logística e um modelo de k-Nearest Neighbors (k-NN). Cada um desses modelos pode capturar diferentes aspectos dos dados. Com o *hard voting*, a classe final será aquela mais votada entre os três modelos. Se o k-NN e a árvore de decisão indicam “doente” e a regressão logística indica “saudável”, a previsão final será “doente”, já que essa foi a maioria.
- **Exemplo 2 — Análise de crédito:** em um sistema de análise de crédito para prever se um cliente é um bom ou mau pagador, podemos usar o *soft voting* com três modelos: Random Forest, Máquinas de Vetores de Suporte (SVM) e uma Rede Neural. Cada modelo fornece uma probabilidade se o cliente pode ser bom ou mau pagador. Se o Random Forest tem mais precisão histórica nesse problema, ele pode receber um peso maior na decisão. A previsão final será a classe com maior média ponderada das probabilidades fornecidas pelos três modelos.

Benefícios do Voting:

O uso de voting permite combinar as previsões de modelos diversos, aproveitando o fato de que diferentes algoritmos capturam diferentes padrões nos dados. Modelos de árvore de decisão, por exemplo, podem ser mais precisos em capturar interações não lineares entre variáveis, enquanto a regressão logística tende a ser mais robusta em relações lineares. Ao combinar modelos com técnicas de voting, o ensemble se torna mais robusto e menos propenso a cometer erros específicos que cada modelo individual poderia cometer.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.5 Stacking

Stacking é uma técnica de ensemble que **combina as previsões de múltiplos modelos (também chamados de modelos de "nível zero") usando um modelo adicional (chamado de "nível 1" ou "meta-modelo") para fazer a previsão final**. Ao contrário de métodos como bagging e boosting, em que as previsões de múltiplos modelos são combinadas diretamente (por votação ou média), **o stacking permite que o modelo de segundo nível aprenda como combinar as previsões dos modelos iniciais de forma mais sofisticada**. Isso é especialmente útil para capturar interações complexas entre as previsões dos modelos de nível zero, resultando em um ensemble mais robusto e preciso. Observe a figura "Intuição do método de *ensemble* por *stacking*".

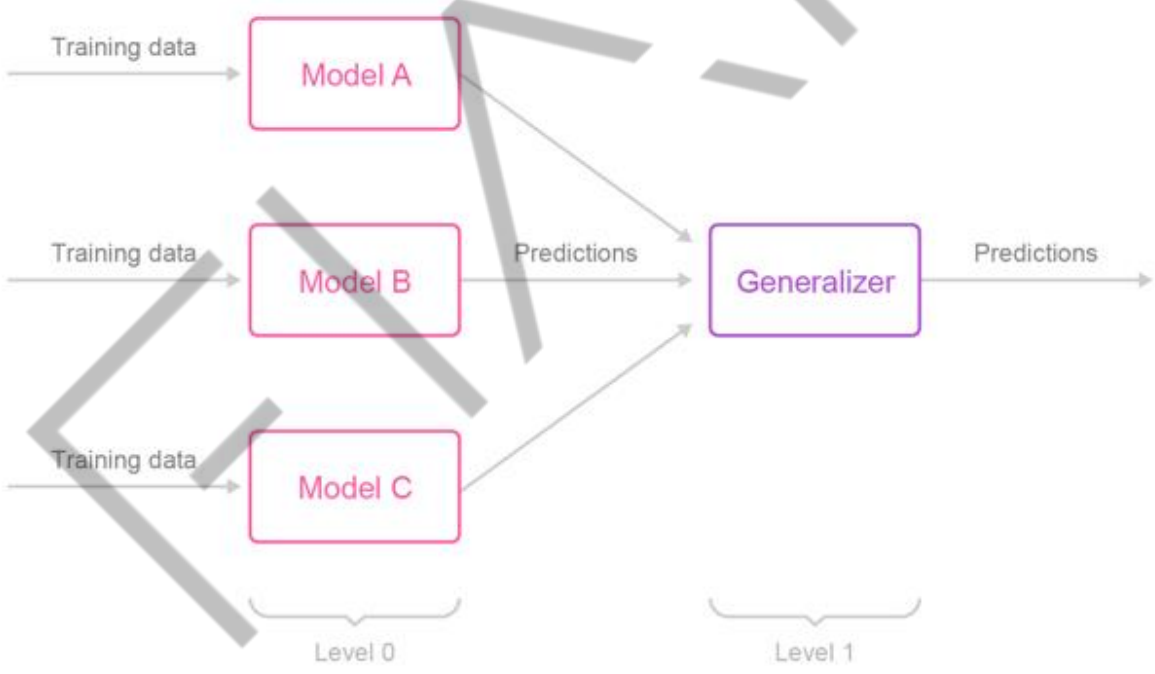


Figura 11 - Intuição do método de *ensemble* por *stacking*
Fonte: Medium (2019), adaptado por FIAP (2024)

Como funciona o Stacking:

1. **Nível Zero (Modelos Base):** vários modelos base são treinados no conjunto de dados original. Esses modelos podem ser de diferentes tipos, como árvores de decisão, regressão logística e k-Nearest Neighbors (k-NN), cada um com suas próprias características e pontos fortes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2. **Geração de Previsões para o Meta-Modelo:** depois que os modelos de primeiro nível são treinados, suas previsões para cada instância no conjunto de dados são salvas. Em vez de fazer previsões diretamente no conjunto de dados de teste, o stacking utiliza as previsões dos modelos de nível zero como features para o próximo modelo. Isso cria um novo conjunto de dados, onde cada coluna representa as previsões de um modelo base.
3. **Nível 1 (Meta-Modelo):** um novo modelo é treinado usando as previsões dos modelos base como entradas. Esse meta-modelo aprende a combinar as previsões iniciais de forma a minimizar o erro final. O meta-modelo é, muitas vezes, um algoritmo simples, como uma regressão linear ou uma árvore de decisão, mas pode ser qualquer modelo que se adapte bem ao problema.

Exemplo de Stacking:

Imagine que queremos prever se um cliente em potencial irá comprar um produto com base em várias variáveis (idade, renda, histórico de navegação etc.). Podemos usar três modelos base: uma regressão logística, uma árvore de decisão e um modelo k-NN. Cada um desses modelos faz suas previsões para cada cliente no conjunto de dados. Em seguida, essas previsões são usadas como features para um meta-modelo, como uma regressão logística, que aprenderá a pesar e combinar essas previsões para produzir a previsão final.

Benefícios do Stacking:

O stacking é especialmente poderoso porque permite explorar a diversidade de diferentes tipos de modelos e, ao mesmo tempo, tirar vantagem de um meta-modelo que aprenda como combinar os pontos fortes de cada um. Assim, enquanto alguns modelos podem ser melhores em capturar certos padrões nos dados, outros podem se sair melhor em diferentes partes do conjunto de dados. O meta-modelo aprende essas diferenças e ajuda a criar um ensemble equilibrado e otimizado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 INTERPRETABILIDADE DE MODELOS

4.1 Qual a importância?

A interpretabilidade dos modelos é um dos temas centrais em Machine Learning, especialmente à medida que os modelos se tornam mais complexos e suas aplicações, mais amplas e críticas. **A interpretabilidade se refere à capacidade de entender como um modelo chega às suas previsões e quais fatores influenciam suas decisões**. Em ambientes de negócios e áreas sensíveis, como saúde, finanças e segurança, essa clareza é essencial para que gestores e profissionais confiem nos modelos e utilizem suas previsões na tomada de decisão. Sem interpretabilidade, um modelo pode ser tratado como uma “caixa-preta”, o que gera desconfiança e resistência, mesmo quando ele for preciso.

A figura a seguir mostra a relação inversa entre performance do modelo e interpretabilidade. Geralmente modelos mais simples levam a um desempenho mais modesto, porém são mais interpretáveis. Outros modelos mais complexos, como SVM ou Redes Neurais, podem não ser diretamente interpretáveis, mas podem levar a um melhor desempenho nas tarefas de Machine Learning.

Figura 12 - Relação inversa entre performance e interpretabilidade de modelos
Fonte: ResearchGate (2022), adaptado por FIAP (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além disso, a interpretabilidade permite identificar possíveis falhas ou vieses nos dados ou nas previsões, tornando mais fácil a detecção de problemas éticos e de justiça. Em muitos casos, decisões importantes, como a concessão de um empréstimo ou o diagnóstico de uma condição médica, não podem ser baseadas apenas em um número ou probabilidade; é necessário entender por que o modelo chegou a esse resultado. Dessa forma, a interpretabilidade não apenas fortalece a adoção dos modelos preditivos, mas também promove uma tomada de decisão mais ética, transparente e fundamentada, garantindo que as organizações possam aproveitar os benefícios da inteligência artificial de forma responsável.

Alguns métodos são inerentemente interpretáveis, como as Árvores de Decisão. Por outro lado, muitos outros não são diretamente interpretáveis. Para estes, é necessário utilizar algumas estratégias para viabilizar a explicabilidade do modelo. As principais estratégias são o ELI5, LIME e SHAP.

4.2 ELI5

A técnica ELI5 (*Explain Like I'm 5*) é uma ferramenta de interpretabilidade que permite entender as previsões de modelos de Machine Learning de forma simples e intuitiva. ELI5 é especialmente útil para explicar modelos complexos, como redes neurais e ensembles, mas também pode ser aplicada a algoritmos mais interpretáveis, como regressão logística e árvores de decisão. A ferramenta oferece explicações detalhadas sobre a importância de cada feature para uma determinada previsão ou para o modelo como um todo, ajudando a identificar quais variáveis estão influenciando o resultado.

Na figura "Exemplo de saída do método ELI5 sobre o dataset de tipos de vinho", temos um exemplo de saída do ELI5. A tarefa do Machine Learning é classificar os tipos de vinho de acordo com suas características. Para cada uma das três classes, o método deve apontar quais são as features que mais contribuíram (em verde) e que menos contribuíram (em vermelho) para determinar se um exemplo pertence àquela classe ou não.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

y=class_0 top features		y=class_1 top features		y=class_2 top features	
Weight[?]	Feature	Weight[?]	Feature	Weight[?]	Feature
+1.359	flavanoids	+0.905	alcohol	+1.020	color_intensity
+1.099	ash	+0.704	hue	+0.627	malic_acid
+0.795	od280/od315_of_diluted_wines	+0.596	proanthocyanins	+0.116	alcalinity_of_ash
+0.706	malic_acid	+0.423	flavanoids	+0.057	ash
+0.110	total_phenols	+0.344	<BIAS>	+0.019	magnesium
+0.098	nonflavanoid_phenols	+0.275	nonflavanoid_phenols	+0.001	proline
+0.015	proline	+0.272	alcalinity_of_ash	-0.058	nonflavanoid_phenols
-0.022	magnesium	+0.241	od280/od315_of_diluted_wines	-0.069	<BIAS>
-0.096	color_intensity	+0.122	total_phenols	-0.318	alcohol
-0.143	hue	+0.007	magnesium	-0.474	hue
-0.325	<BIAS>	-0.014	proline	-0.645	total_phenols
-0.450	proanthocyanins	-0.795	ash	-0.729	proanthocyanins
-0.519	alcohol	-1.072	malic_acid	-1.200	od280/od315_of_diluted_wines
-0.610	alcalinity_of_ash	-1.875	color_intensity	-1.902	flavanoids

Figura 13 - Exemplo de saída do método ELI5 sobre o dataset de tipos de vinho
Fonte: coderzcolumn (2022)

Com o ELI5, é possível **visualizar os pesos das features** (para modelos lineares), as contribuições de cada variável em uma previsão específica, e as interações que estão impactando as decisões do modelo. Por exemplo, ao usar o ELI5 em uma regressão logística, a ferramenta exibe os coeficientes das variáveis, facilitando o entendimento de como cada uma contribui positivamente ou negativamente para a previsão.

Essa técnica é especialmente valiosa para tornar os modelos mais transparentes e aumentar a confiança dos usuários, pois traduz as previsões em informações interpretáveis e fáceis de entender, mesmo para pessoas sem conhecimentos profundos em Machine Learning. Dessa forma, ELI5 contribui para uma adoção mais ampla e consciente dos modelos de IA na tomada de decisão.

4.3 LIME

O método LIME (Local Interpretable Model-agnostic Explanations) é uma técnica poderosa de interpretabilidade que permite entender previsões de modelos de Machine Learning ao explicar cada instância individualmente. **Ao contrário de métodos que tentam interpretar o modelo como um todo, o LIME foca na explicação de previsões locais, ou seja, ele explica a decisão do modelo para uma única instância ou caso específico**. Essa abordagem é especialmente útil em modelos complexos e de “caixa-preta”, como redes neurais e ensembles, onde é difícil compreender diretamente como cada variável influencia a saída.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura “Exemplo de saída do método LIME sobre o dataset de qualidade de vinhos” mostra uma saída típica do LIME. Aqui o objetivo é determinar, através das características do vinho, se é um vinho “bom” ou “ruim”. Para esta instância em particular, o modelo preditivo a classificou como “bom”, com 96% de certeza, ainda que com 4% de certeza que poderia ser um vinho “ruim”. A parte central da figura mostra as forças de interação de cada variável e seus *thresholds* (pontos de corte). O quadro à direita indica como as features contribuíram para a classificação (laranja contribui para ser um vinho “bom” e azul contribui para ser um vinho “ruim”). O método aponta que um vinho é considerado bom quando o teor alcóolico está acima de 11,40 (no caso da instância é 13), e o nível de cloreto deve ser menor ou igual à 0,04 (na instância foi 0,04).

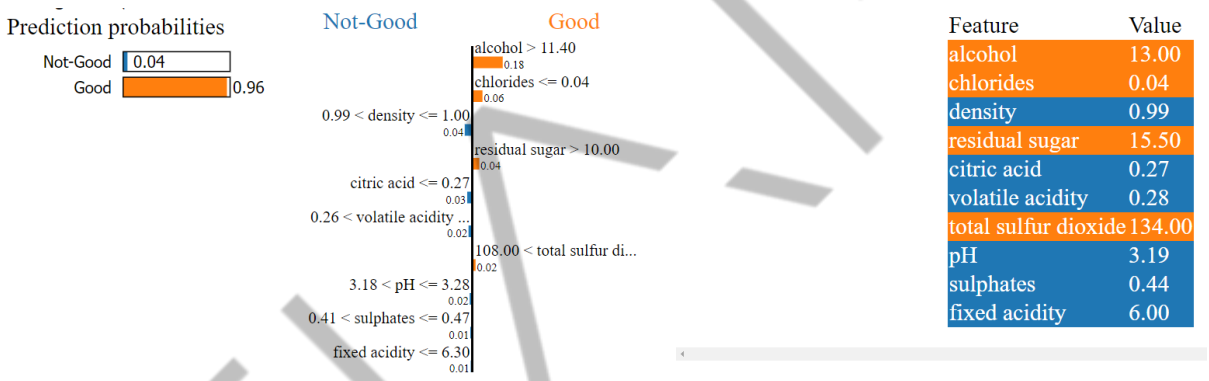


Figura 14 - Exemplo de saída do método LIME sobre o dataset de qualidade de vinhos
Fonte: O'Fallon Labs (2023)

O LIME funciona perturbando ligeiramente os dados de entrada da instância a ser explicada e observando como o modelo reage a essas mudanças. Com base nessa análise, ele gera um modelo interpretável (geralmente uma regressão linear ou uma árvore de decisão) que simula o comportamento do modelo original na vizinhança da instância analisada. Isso permite identificar quais variáveis foram mais relevantes para aquela previsão específica e de que maneira elas influenciaram o resultado.

Esse método é muito valioso para tomar decisões baseadas em IA, pois ajuda a responder perguntas do tipo “por que o modelo fez essa previsão?”, proporcionando uma visão clara e intuitiva para usuários que precisam confiar nas previsões. Ao fornecer uma explicação local e específica, o LIME torna os modelos mais transparentes e facilita a identificação de padrões ou anomalias em instâncias

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

individuais, contribuindo para uma melhor compreensão e confiança na inteligência artificial.

4.4 SHAP

O método SHAP (SHapley Additive exPlanations) é uma técnica de interpretabilidade baseada na teoria dos jogos que **oferece explicações detalhadas sobre o impacto de cada variável nas previsões de um modelo de Machine Learning**. Inspirado nos valores de Shapley, usados em teoria dos jogos para distribuir recompensas de forma justa entre jogadores que contribuem para um resultado, o SHAP calcula a “contribuição” de cada feature para a previsão final. Dessa forma, ele atribui a cada variável um valor numérico que representa sua influência positiva ou negativa sobre a decisão do modelo. Veja o exemplo da figura a seguir.

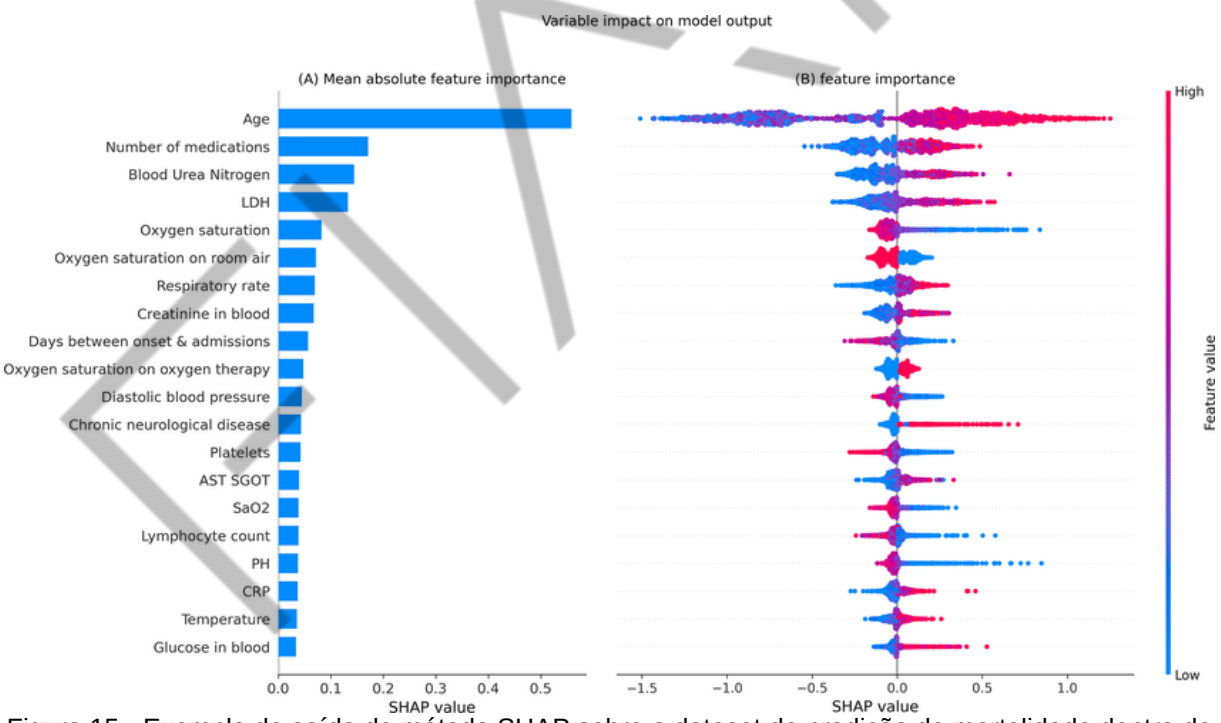


Figura 15 - Exemplo de saída do método SHAP sobre o dataset de predição de mortalidade dentro de 21 dias em decorrência de COVID-19
Fonte: ResearchGate (2021)

A figura “Exemplo de saída do método SHAP sobre o dataset de predição de mortalidade dentro de 21 dias em decorrência de COVID-19” ilustra o diagrama SHAP de um modelo treinado para reconhecer pessoas que podem evoluir a óbito em até

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

21 dias em decorrência de COVID-19. **O gráfico é complexo e nos mostra várias informações!** Vamos por partes:

- A parte (A) ilustra a importância geral das features.
- A parte (B) ilustra de que forma cada feature contribui para a predição.
- A cor de cada ponto de dados representa o valor da feature, sendo que o vermelho corresponde a valores altos e o azul a valores baixos.
- Valores de SHAP acima de 0 sugerem associação positiva com o desfecho, ou seja, contribuem para indicar que o paciente morrerá.
- Valores de SHAP abaixo de 0 sugerem associação negativa com o desfecho, ou seja, contribuem para indicar que o paciente sobreviverá.

Desta forma, percebe-se que os fatores mais importantes para predizer a mortalidade do paciente com COVID-19 nesta base é a **idade do paciente**, **quantos medicamentos administra** e uma **porção de indicadores de saúde**.

No caso da “idade”, quanto menor (mais azul), mais o SHAP tenderia para o negativo, portanto, maior a chance de sobrevivência do paciente. Por outro lado, quanto maior a idade (mais vermelho), mais o SHAP tenderia para o positivo, indicando uma possível morte em até 21 dias.

Observe também o caso de saturação de oxigênio (*oxygen saturation*). De modo simplificado, este índice mede a capacidade da pessoa respirar. Quanto menor o nível de saturação (valores em azul), maior a chance de a pessoa vir a óbito.

Agora você pode continuar interpretando o gráfico por conta própria!

O grande diferencial do SHAP é sua capacidade de fornecer explicações consistentes e globais para o comportamento do modelo, ao mesmo tempo em que permite analisar previsões individuais. Ao calcular a importância de cada variável em relação às demais, ele gera explicações que são tanto locais (explicando uma única previsão) quanto globais (explicando tendências gerais do modelo). Isso é especialmente útil para modelos complexos, como ensembles e redes neurais, onde é difícil identificar diretamente o papel de cada variável.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O SHAP é amplamente adotado por sua precisão e consistência na interpretação dos modelos, pois suas explicações são intuitivas e matematicamente bem fundamentadas. Isso o torna uma ferramenta valiosa para aumentar a transparência e a confiabilidade dos modelos de Machine Learning, permitindo que stakeholders e usuários compreendam melhor os fatores que influenciam cada previsão e tomem decisões mais informadas com base em insights interpretáveis.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 SIMPLIFICANDO A CRIAÇÃO DE MODELOS PREDITIVOS

5.1 Facilitando o acesso à ciência de dados

Ferramentas de Machine Learning como soluções **low code** e **no code** tornaram os modelos preditivos mais acessíveis, permitindo que profissionais de várias áreas experimentem técnicas preditivas sem programação avançada.

O **PyCaret** é uma biblioteca de Python que simplifica tarefas como pré-processamento de dados, modelagem e ajuste de hiperparâmetros, facilitando o desenvolvimento de soluções em poucas linhas de código. Na sequência, exploraremos o uso desta ferramenta.

Figura 16 - PyCaret, uma das principais bibliotecas para gerar modelos de Machine Learning de forma simplificada com pouca programação
Fonte: Pycaret.org (2024)

Embora essas ferramentas acelerem o processo e tornem o Machine Learning mais acessível, o conhecimento em ciência de dados e algoritmos ainda é essencial para garantir a eficácia e confiabilidade dos modelos. O domínio dos conceitos fundamentais permite escolher e ajustar corretamente os modelos, evitando implementações superficiais ou inadequadas.

5.2 Vantagens e Desvantagens

Como toda ferramenta ou tecnologia, as aplicações low code ou no code para desenvolvimento de modelos de aprendizado de máquina trazem vantagens e desvantagens.

Listamos as principais no quadro a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vantagens do PyCaret ou outras ferramentas low code	Desvantagens do PyCaret ou outras ferramentas low code
Redução do tempo de desenvolvimento: automatizam tarefas como pré-processamento, modelagem, tuning de hiperparâmetros e avaliação, acelerando a construção de modelos preditivos.	Limitação na customização e controle fino: estrutura pré-definida que pode dificultar ajustes específicos, limitando as opções de personalização do usuário.
Facilidade para experimentação: permitem que profissionais com pouca experiência em programação experimentem a ML, democratizando o acesso às técnicas preditivas.	Risco de superficialidade: o fácil acesso pode resultar em implementações sem a devida compreensão dos algoritmos, levando a modelos mal ajustados.
Padronização do pipeline e comparação de modelos: simplifica a comparação de várias abordagens, ajudando a identificar o modelo mais adequado ao problema.	Dependência da ferramenta: limita o aprendizado técnico, reduzindo a exposição ao código e ao funcionamento interno dos métodos de Machine Learning.

Quadro 1 - Vantagens e desvantagens do PyCaret ou outras ferramentas low code
Fonte: Elaborado pelo autor (2024)

Dica: nossa principal dica é para conhecer e utilizar tais ferramentas, mas não dependa exclusivamente delas para treinar e avaliar seus modelos preditivos!

5.3 Introdução ao PyCaret

5.3.1 Usos gerais e como se aprofundar na ferramenta

O PyCaret é uma ferramenta poderosa e prática para quem deseja aplicar Machine Learning de maneira rápida e eficiente. Seu uso é especialmente indicado para tarefas que envolvem experimentação, prototipagem e comparação de modelos, pois permite realizar desde a preparação de dados até a criação e ajuste de modelos preditivos com uma quantidade mínima de código.

Para explorar todo o potencial do PyCaret, é útil conhecer as várias áreas de modelagem que ele suporta. O PyCaret dá suporte a quatro principais tipos de modelagem:

1. **Modelagem Supervisionada:**

   a. **Classificação:** o PyCaret oferece uma ampla gama de algoritmos para resolver problemas de classificação, como árvores de decisão, regressão logística, k-Nearest Neighbors (kNN), SVM (Support Vector Machine), Random Forests, XGBoost, entre outros.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

   b. **Regressão:** para problemas de regressão, o PyCaret disponibiliza modelos como Regressão Linear, Lasso, Ridge, Random Forest Regressor, e Gradient Boosting Regressor.

2. **Modelagem Não Supervisionada:**
   a. **Clusterização:** em cenários onde não há rótulos nos dados, o PyCaret permite realizar tarefas de clusterização usando algoritmos como K-Means, Mean Shift, e DBSCAN.
   b. **Redução de Dimensionalidade:** o PyCaret também facilita a aplicação de métodos de redução de dimensionalidade, como PCA (Principal Component Analysis), para simplificar os dados e torná-los mais interpretáveis sem perder informações relevantes.
3. **Modelagem com Séries Temporais:** o PyCaret oferece suporte para análise de séries temporais, facilitando o desenvolvimento de previsões baseadas em dados com dependência temporal, como demanda futura de produtos, variação de preços e análise de tendência.
4. **Modelagem com Dados de Texto:** o PyCaret também tem suporte para NLP (Natural Language Processing) e permite criar pipelines automáticos para tarefas de processamento de linguagem natural, como classificação de texto e análise de sentimentos.

A seguir veremos um exemplo de como utilizar o pipeline do PyCaret.

Dica: consulte a documentação do PyCaret! Não deixe de ver os tutoriais das várias formas de modelagem suportadas: <https://pycaret.gitbook.io/docs/get-started/tutorials>.

5.3.2 Hands On: Modelagem Preditiva via PyCaret

Nesse primeiro Hands On, vamos ver como utilizar o poder do PyCaret para avaliar múltiplos algoritmos de **regressão** para o mesmo problema. Para exemplos de outras formas de modelagem, não deixe de consultar os tutoriais da ferramenta!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vale mencionar que essa prática também será feita no Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5 – Sklearn), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Sobre a base:

Utilizaremos a famosa base de dados de custos de seguro de saúde nos EUA mediante as características do segurado.

Dica: utilize o arquivo de apoio “insurance.csv”.

Código:

O primeiro passo é a instalação da biblioteca! Feito isso, precisamos importar o código necessário, leitura e exibição dos dados (ou de uma parte deles).

```
!pip install pycaret

# Imports
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

import pycaret
from pycaret.regression import *

df = pd.read_csv("insurance.csv")
df.sample(10)
```

Código-fonte 1 - Instalação da biblioteca, imports e leitura do dataset
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	age	sex	bmi	children	smoker	region	charges
355	46	male	27.600	0	no	southwest	24603.04837
746	34	male	27.000	2	no	southwest	11737.84884
205	28	female	28.880	1	no	northeast	4337.73520
1065	42	female	25.300	1	no	southwest	7045.49900
493	61	male	43.400	0	no	southwest	12574.04900
450	39	male	29.600	4	no	southwest	7512.26700
116	58	male	49.060	0	no	southeast	11381.32540
1146	60	male	32.800	0	yes	southwest	52590.82939
566	38	female	40.565	1	no	northwest	6373.55735
1062	59	male	41.140	1	yes	southeast	48970.24760

Figura 17 - Saída do código-fonte 1
Fonte: Elaborado pelo autor (2024)

Agora é hora de dividir os dados em **treino** e **teste**, bem como inicializar o PyCaret (fazer o `setup()`). Esta inicialização é justamente criar um experimento. Toda execução do PyCaret é um experimento. No caso, estamos inicializando o experimento indicando que os dados de treinamento serão usados, para predizer a coluna “charges”, utilizando 70% dos dados como **treino** e 30% como **validação**. Notem que o **teste** ficou intacto e apartado do experimento.

```
     # Divisão de treino e teste
     train, test = train_test_split(df,
                                    test_size=0.2,
                                    random_state=42)

     # IMPORTANTE: o pycaret sempre começa com o setup()
     reg = setup(data=train,          # Dados para treino
                 target="charges",    # Label a ser predito
                 train_size=0.7)      # Proporção de dados para
treino (o resto é validação)
```

Código-fonte 2 - Divisão de TREINO e TESTE e setup do experimento com PyCaret
Fonte: Elaborado pelo autor (2024)

A figura “Saída parcial do código-fonte 2” mostra o resultado deste setup. Note que já temos uma série de manipulações dos dados sendo feitas de forma automática, como imputação de dados numéricos e categóricos! Outros pré-processamentos

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

também incluem OneHotEncoder e esquema de avaliação com K-Fold (omitido da imagem).

	Description	Value
0	Session id	5043
1	Target	charges
2	Target type	Regression
3	Original data shape	(1070, 7)
4	Transformed data shape	(1070, 10)
5	Transformed train set shape	(749, 10)
6	Transformed test set shape	(321, 10)
7	Numeric features	3
8	Categorical features	3
9	Preprocess	True
10	Imputation type	simple
11	Numeric imputation	mean
12	Categorical imputation	mode

Figura 18 - Saída parcial do código-fonte 2
Fonte: Elaborado pelo autor (2024)

Agora **podemos comparar diversos algoritmos de regressão com um simples comando**. O próximo código pede para que tal comparação seja feita, ordenando os melhores modelos pelo R^2. Outras métricas de regressão poderiam ser escolhidas.

```
# Agora vamos comparar diversos modelos de regressão
compare_models(sort="R2")
```

Código-fonte 3 - Comparando algoritmos de regressão e ordenando pelo R^2
Fonte: Elaborado pelo autor (2024)

O resultado está na figura “Saída do código-fonte 3”. Ela lista todos os modelos de regressão implementados pelo PyCaret e suas avaliações na VALIDAÇÃO sob diversas métricas. O melhor modelo (gbr – Gradient Boosting Regressor) foi escolhido por ter o melhor R^2.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
gbr	Gradient Boosting Regressor	2804.2743	24203959.2790	4822.0090	0.8253	0.4806	0.3447	0.3550
rf	Random Forest Regressor	2935.4257	25872215.9444	5004.3304	0.8133	0.4924	0.3634	0.4110
ada	AdaBoost Regressor	3904.4583	26118679.2632	5041.2557	0.8119	0.5650	0.5896	0.2260
lightgbm	Light Gradient Boosting Machine	3093.1794	26872500.9352	5083.4238	0.8065	0.5813	0.3774	0.3820
et	Extra Trees Regressor	2939.5497	28122377.5883	5211.9447	0.7983	0.4906	0.3357	0.4070
xgboost	Extreme Gradient Boosting	3202.1637	30409690.8000	5427.5043	0.7830	0.5984	0.4177	0.1590
ridge	Ridge Regression	4368.3995	38907202.9003	6197.6773	0.7190	0.5718	0.4271	0.0950
br	Bayesian Ridge	4361.3543	38909178.8874	6197.3098	0.7189	0.5788	0.4259	0.1490
llar	Lasso Least Angle Regression	4352.6352	38905384.4524	6196.2950	0.7189	0.5930	0.4245	0.1040
lar	Least Angle Regression	4352.2313	38907992.5432	6196.4927	0.7189	0.5968	0.4244	0.1010
lasso	Lasso Regression	4352.6324	38905358.6490	6196.2930	0.7189	0.5931	0.4245	0.1350
lr	Linear Regression	4352.2313	38907992.5432	6196.4927	0.7189	0.5968	0.4244	1.0630
dt	Decision Tree Regressor	3267.1860	44584227.6788	6551.4725	0.6826	0.5611	0.3823	0.1030
huber	Huber Regressor	3538.2231	50546676.3887	7024.5287	0.6344	0.5011	0.2250	0.2370
par	Passive Aggressive Regressor	4419.2452	68575232.6848	8256.6815	0.5173	0.5044	0.2547	0.2180
en	Elastic Net	7421.8058	90973634.7485	9528.7186	0.3602	0.7305	0.9093	0.0950
knn	K Neighbors Regressor	8223.8487	134443791.2000	11534.0252	0.0659	0.8452	0.9531	0.1970
omp	Orthogonal Matching Pursuit	9207.0882	135472095.9554	11626.3049	0.0500	0.8703	1.1462	0.1060
dummy	Dummy Regressor	9077.1424	145512722.4000	12023.2622	-0.0067	0.9664	1.4374	0.1170

Figura 19 - Saída do código-fonte 3
Fonte: Elaborado pelo autor (2024)

Podemos inclusive obter apenas este modelo, se desejarmos:

```
# Crie o melhor modelo
gbr = create_model("gbr")
```

Código-fonte 4 - Criando um único modelo baseado em Gradient Boosting
Fonte: Elaborado pelo autor (2024)

O qual está ilustrado na figura “Saída do código-fonte 4”. Notem o esquema de avaliação em 10-Fold cross-validation, trazendo ainda mais robustez ao método.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Fold	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	2152.2677	13195593.5491	3632.5740	0.9213	0.4573	0.4054
1	2839.7382	23534954.7129	4851.2838	0.8142	0.4961	0.3756
2	3686.4230	41955834.7029	6477.3324	0.7338	0.5489	0.2991
3	2822.7007	27082586.5873	5204.0933	0.7104	0.5362	0.2978
4	2931.7011	25158920.0573	5015.8668	0.8425	0.4420	0.3073
5	2790.3005	19539685.2358	4420.3716	0.8768	0.4124	0.3564
6	3456.0026	39269753.8816	6266.5584	0.7382	0.5074	0.3674
7	2079.3824	14218696.6827	3770.7687	0.8995	0.4791	0.3629
8	3142.1959	25865684.3101	5085.8317	0.7892	0.4888	0.3740
9	2142.0312	12217883.0704	3495.4089	0.9274	0.4382	0.3012
Mean	2804.2743	24203959.2790	4822.0090	0.8253	0.4806	0.3447
Std	521.9487	9699616.7497	975.8017	0.0766	0.0416	0.0375

Figura 20 - Saída do código-fonte 4
Fonte: Elaborado pelo autor (2024)

É possível, ainda, otimizar o modelo desenvolvido:

```
# Como tunar um modelo no próprio pycaret?
tuned_gbr = tune_model(gbr)
```

Código-fonte 5 - Tunando o modelo baseado em Gradient Boosting
Fonte: Elaborado pelo autor (2024)

Fold	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	2478.1444	15366699.0168	3920.0381	0.9084	0.5087	0.4843
1	2959.5666	24726376.1952	4972.5623	0.8048	0.4993	0.3756
2	3703.1469	40050211.2064	6328.5236	0.7459	0.5619	0.3239
3	2837.5216	26856612.5922	5182.3366	0.7128	0.5631	0.3287
4	3023.6968	25703028.6639	5069.8154	0.8391	0.4650	0.3080
5	2979.0186	22712495.4411	4765.7628	0.8568	0.4273	0.3699
6	3575.3321	37348909.4780	6111.3754	0.7510	0.4739	0.3464
7	2128.0214	14039872.8045	3746.9818	0.9007	0.4412	0.2984
8	3022.4193	25053256.5695	5005.3228	0.7958	0.4979	0.3275
9	2333.0766	12477090.9797	3532.2926	0.9259	0.5034	0.3653
Mean	2903.9944	24433455.2947	4863.5012	0.8241	0.4942	0.3528
Std	473.3135	8687890.7849	883.0695	0.0705	0.0427	0.0503

Figura 21 - Saída do código-fonte 5
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vale mencionar que o PyCaret facilitou bastante a criação de gráficos de avaliação do modelo. Agora vamos avaliar os ERROS e na sequência a **importância de features**:

```
# Comparar predições e resultados
plot_model(tuned_gbr, plot="error")

# Quais são as features mais importantes?
plot_model(tuned_gbr, plot="feature")
```

Código-fonte 6 - Plotando detalhes do modelo de Gradient Boosting tunado
Fonte: Elaborado pelo autor (2024)

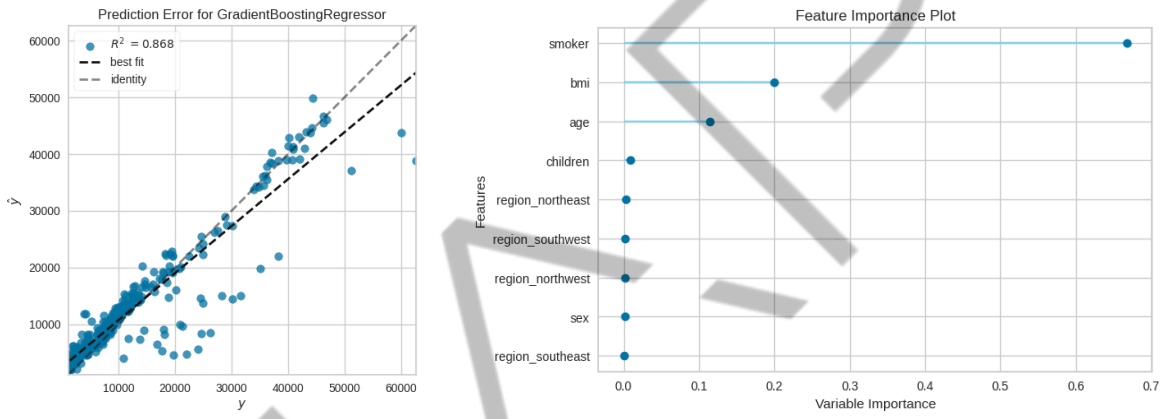


Figura 22 - Saídas do código-fonte 6
Fonte: Elaborado pelo autor (2024)

Dica: esses são apenas dois exemplos de gráficos que o PyCaret permite gerar de forma facilitada! Não deixe de consultar os tutoriais e a documentação para aprender mais.

Vale lembrar que as métricas reportadas são na VALIDAÇÃO, uma parte reservada do TREINO. Para coletar as métricas no TESTE, precisamos pedir para o PyCaret gerar as predições:

```
# Fazendo predições no TESTE
predict_model(tuned_gbr, data=test)
```

Código-fonte 7 - Predizendo os exemplos do teste
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Gradient Boosting Regressor	2530.6761	19303625.0508	4393.5891	0.8757	0.4244	0.3289

	age	sex	bmi	children	smoker	region	charges	prediction_label
764	45	female	25.174999	2	no	northeast	9095.068359	11343.505348
887	36	female	30.020000	0	no	northwest	5272.175781	5221.777013
890	64	female	26.885000	0	yes	northwest	29330.982422	28231.108298
1293	46	male	25.745001	3	no	northwest	9301.893555	10332.892622
259	19	male	31.920000	0	yes	northwest	33750.292969	33809.115563
...	...	...	...	...	...	...	...	...
109	63	male	35.090000	0	yes	southeast	47055.531250	46364.769373
575	58	female	27.170000	0	no	northwest	12222.898438	13995.886103
535	38	male	28.025000	1	no	northeast	6067.126953	7417.007091
543	54	female	47.410000	0	yes	southeast	63770.429688	48819.974265
846	51	female	34.200001	1	no	southwest	9872.701172	10226.392262

268 rows × 8 columns

Figura 23 - Saídas do código-fonte 7
Fonte: Elaborado pelo autor (2024)

Baseado nas colunas “charges” e “prediction_label” do dataframe da figura “Saídas do código-fonte 7”, podemos calcular as métricas pertinentes desejadas. Uma vez que o processo está “pronto” é possível **finalizar o modelo**, retreinando-o com todos os dados de TREINO + TESTE, ou seja, usando os mesmos hiperparâmetros para treinar o modelo com todos os dados. Da seguinte forma:

```
    # Finalizar o modelo, criando a versão completa treinada
em TODOS os dados
    gbr_final = finalize_model(tuned_gbr)
```

Código-fonte 8 - Finalizando o modelo (retreino com o dataset inteiro)
Fonte: Elaborado pelo autor (2024)

O código-fonte “Finalizando o modelo (retreino com o dataset inteiro)” não gera saída visível, porém, neste ponto, o objeto `gbr_final` contém o regressor baseado em Gradient Boosting treinado com todos os dados do dataset. Este objeto pode ser utilizado para exportar o modelo, ou fazer predições com novos dados! Recomendamos olhar tutoriais da ferramenta para melhor compreensão de como esta última parte conversa com MLOps. **Portanto, com poucas linhas de código, conseguimos comparar diversos algoritmos de Machine Learning e explorar o melhor deles**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 APRENDIZADO POR REFORÇO

6.1 Introdução

O aprendizado por reforço (*Reinforcement Learning*, RL) é uma abordagem única de Machine Learning que **permite a um agente aprender a tomar decisões em um ambiente dinâmico, buscando maximizar uma recompensa acumulada**. O processo é muito similar ao adestramento canino: o cão recebe um estímulo positivo (um petisco, por exemplo) após realizar uma ação desejada, como sentar ou deitar, reforçando o comportamento correto e incentivando-o a repeti-lo (figura “Analogia do aprendizado por reforço”).

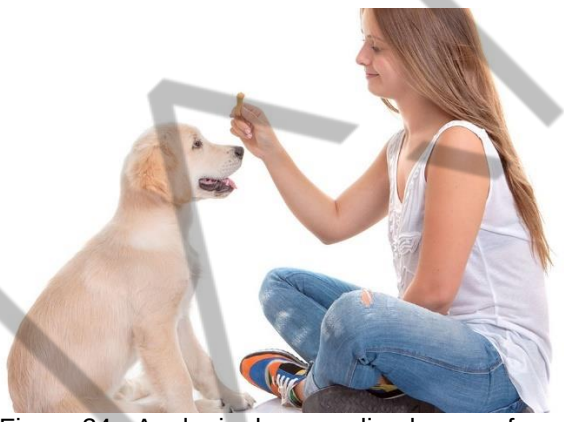


Figura 24 - Analogia do aprendizado por reforço
Fonte: Adestramento de cães (2024)

Diferente do aprendizado supervisionado, onde o modelo aprende com base em exemplos de entrada e saída fornecidos, ou do aprendizado não supervisionado, onde descobre padrões em dados não rotulados, **o aprendizado por reforço permite que o agente aprenda com suas próprias experiências e interações**. Isso é particularmente útil para situações em que é necessário adaptar-se a um ambiente em constante mudança, ou onde as ações influenciam os próximos estados do sistema. O RL se mostra essencial para resolver problemas complexos, como jogar xadrez, dirigir veículos autônomos, gerenciar sistemas de recomendação e otimizar operações industriais, pois ensina o agente a “aprender fazendo”, ajustando suas estratégias com base nos feedbacks que recebe.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.2 Componentes e Aplicações

Os principais componentes do aprendizado por reforço estão ilustrados na figura “Componentes do aprendizado por reforço”.

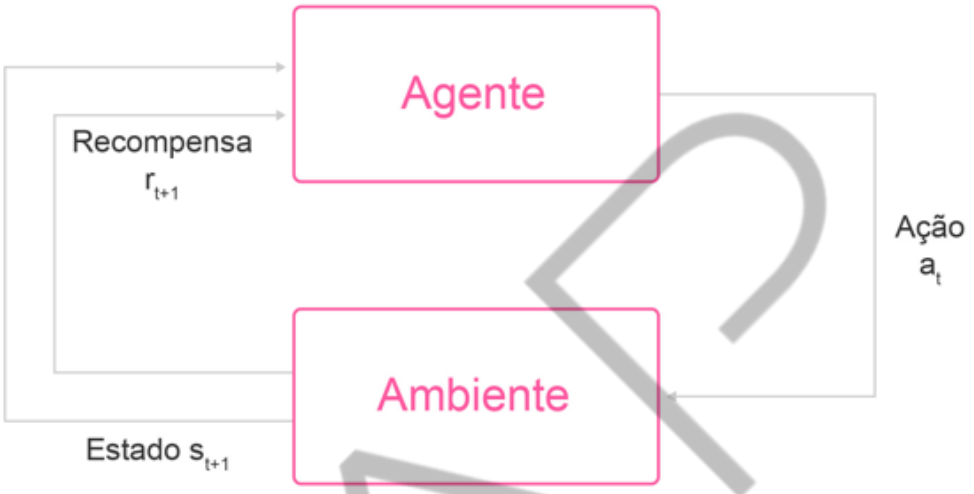


Figura 25 - Componentes do aprendizado por reforço
Fonte: ResearchGate (2019), adaptado por FIAP (2024)

São eles:

- **Agente:** a entidade que toma ações no ambiente. Seu objetivo é aprender uma política que maximize a recompensa acumulada ao longo do tempo.
- **Ambiente:** o contexto no qual o agente interage e toma decisões. O ambiente responde às ações do agente, fornecendo novos estados e recompensas.
- **Ação:** as decisões ou movimentos que o agente pode tomar em um dado estado.
- **Estado:** a representação da situação atual do agente no ambiente. Um estado é atualizado com base na ação realizada pelo agente e representa informações sobre o ambiente.
- **Recompensa:** o feedback que o agente recebe após cada ação, indicando se sua decisão foi positiva ou negativa em relação ao objetivo final.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esses componentes formam a base para uma **ampla gama de aplicações**. Por exemplo:

- Em **jogos**, como xadrez ou Go, o agente aprende estratégias vencedoras ao testar movimentos e avaliar o impacto de suas ações no resultado.
- Em **robótica**, o RL permite que robôs aprendam tarefas complexas, como caminhar ou manipular objetos, ao receber recompensas por ações bem-sucedidas que os aproximam de seu objetivo.
- Em **finanças**, o RL é usado para otimizar carteiras de investimento, onde o agente ajusta alocações com base nos retornos do mercado.

Outras aplicações incluem controle de tráfego, automação industrial e até saúde, onde o aprendizado por reforço auxilia no planejamento de tratamentos personalizados.

6.3 Funcionamento

No RL, o agente aprende por tentativa e erro, testando diferentes ações e ajustando suas escolhas com base nas recompensas obtidas. Esse processo é chamado de **exploração** (*exploration*: testar ações para obter mais informações) e **explotação** (*exploitation*: escolher ações que já são conhecidas por maximizar a recompensa). Inicialmente, o agente explora o ambiente, experimentando várias ações e observando os resultados. À medida que acumula conhecimento, ele começa a explotar ações que têm mais probabilidade de gerar recompensas positivas. A chave do bom funcionamento e convergência é encontrar um bom equilíbrio entre ambas as abordagens. Observe a figura “Técnicas intercaláveis necessárias ao aprendizado por reforço”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 26 - Técnicas intercaláveis necessárias ao aprendizado por reforço
Fonte: bizzdesign (2015)

O aprendizado por reforço utiliza funções de valor para estimar as recompensas futuras esperadas, o que ajuda o agente a tomar decisões otimizadas. A política do agente, que pode ser aprendida com métodos como **Q-learning** e **Deep Q-learning**, define as ações que ele deve tomar em diferentes estados para maximizar a recompensa. Métodos avançados, como políticas baseadas em redes neurais, permitem que agentes em ambientes complexos tomem decisões mais informadas, mesmo em situações em que o número de estados possíveis é muito grande.

Algoritmo Q-Learning:

O algoritmo Q-Learning é uma técnica de aprendizado por reforço que permite a um agente aprender a tomar decisões ótimas em um ambiente através de um processo de tentativa e erro. Ele funciona ao atualizar uma tabela, chamada Q-table, que armazena os valores de recompensa esperados para cada par de estado e ação.

Imagine que um agente está preso em um labirinto e precisa aprender como sair dele encontrando o caminho mais curto. Os passos do algoritmo são:

1. **Inicializar a Q-table:** comece criando uma Q-table com todos os pares de estado e ação possíveis. Cada célula da Q-table começa com um valor inicial (geralmente zero), representando a recompensa esperada para cada ação em cada estado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2. **Escolher uma ação (*Exploration vs. Exploitation*):** no estado atual, o agente precisa escolher uma ação. Ele pode explorar o ambiente escolhendo uma ação aleatória (exploração) ou optar pela ação que já oferece a maior recompensa estimada na Q-table (explotação). Esse equilíbrio entre exploração e explotação permite que o agente aprenda sobre o ambiente ao mesmo tempo que otimiza suas decisões.

3. **Executar a ação e observar a recompensa:** o agente executa a ação escolhida, o que o leva a um novo estado do labirinto. Com base na ação, ele recebe uma recompensa (por exemplo, uma recompensa positiva caso se aproxime da saída do labirinto e uma negativa caso se colida com uma parede ou faça um movimento não produtivo).

4. **Atualizar a Q-table:** o valor Q para o par de estado e ação é atualizado usando a fórmula de atualização do Q-Learning:

$$\boldsymbol{Q(s,a) = Q(s,a) + \alpha(r + \gamma \max Q(s',a') - Q(s,a))}$$

Em que:

- $\boldsymbol{s}$ é o estado atual e $\boldsymbol{a}$ é a ação escolhida.
- $\boldsymbol{s'}$ é o novo estado após a ação e $\boldsymbol{a'}$ são ações possíveis em $\boldsymbol{s'}$.
- $\boldsymbol{r}$ é a recompensa recebida ao tomar a ação $\boldsymbol{a}$.
- $\boldsymbol{\alpha}$ é a taxa de aprendizado, que controla o peso da nova informação.
- $\boldsymbol{\gamma}$ é o fator de desconto, que define a importância das recompensas futuras em relação às recompensas imediatas.

Esse processo ajusta o valor na Q-table para refletir a recompensa atual e as recompensas futuras esperadas para o par $(\boldsymbol{s,a})$.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5. **Repetir até o agente aprender:** o agente repete os passos 2 a 4 para múltiplas interações no ambiente até que a Q-table converge para valores estáveis. Com o tempo, o agente aprende a escolher ações que maximizam a recompensa total, encontrando o caminho mais eficiente para a saída do labirinto.

Após várias iterações, a Q-table do agente conterá as recompensas estimadas para todas as combinações de estado e ação, permitindo que ele tome decisões otimizadas. Assim, ao seguir o Q-Learning, o agente aprende uma política ideal para alcançar o objetivo desejado — neste caso, encontrar a saída do labirinto da forma mais eficiente possível.

Portanto, o RL é uma poderosa abordagem de aprendizado autônomo, capacitando agentes a lidar com ambientes complexos e realizar tarefas sofisticadas ao se basear em seu próprio aprendizado e adaptação. Sua utilização, no entanto, deve fazer sentido no contexto de aplicação, a fim de evitar problemas éticos ou erros grosseiros pelo uso inicial do sistema (ainda não devidamente calibrado).

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 HANDS ON

Agora que passamos pelos tópicos avançados de Machine Learning, está na hora de os implementar. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

7.1 Sobre a base de dados

Nesta prática, utilizaremos uma base de dados comumente empregada para prever a saída (*churn*) de clientes de uma instituição bancária com base em características demográficas e comportamentais. As features disponibilizadas são:

- **RowNumber:** número da linha no dataset.
- **CustomerId:** ID do cliente no banco.
- **Surname:** sobrenome do cliente.
- **CreditScore:** pontuação de crédito do cliente. Esse valor indica a confiabilidade financeira do cliente, com pontuações mais altas indicando menor risco.
- **Geography:** país de residência do cliente (por exemplo: França, Espanha, Alemanha).
- **Gender:** gênero do cliente (masculino ou feminino).
- **Age:** idade do cliente.
- **Tenure:** tempo (em anos) que o cliente está com o banco.
- **Balance:** saldo na conta bancária do cliente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **NumOfProducts:** número de produtos bancários (contas, cartões, empréstimos etc.) que o cliente possui no banco.
- **HasCrCard:** indica se o cliente possui um cartão de crédito (1 = sim, 0 = não).
- **IsActiveMember:** indica se o cliente é um membro ativo (1 = sim, 0 = não).
- **EstimatedSalary:** salário estimado do cliente.
- **Exited:** variável alvo que indica se o cliente deixou o banco (1 = sim, 0 = não). Essa coluna é a saída a ser prevista pelo modelo de *churn*.

Dica: utilize o arquivo de apoio "churn_modelling.csv".

7.2 Preparação da base

O primeiro passo é a **importação do código necessário e criação de funções de apoio para calcular algumas métricas e exibir a matriz de confusão.**

```
# Instalações
!pip install lime
!pip install shap

# Suprimir warnings
import warnings
warnings.filterwarnings('ignore')

# Leitura de dados e gráficos
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Preparação de dados
from imblearn.under_sampling import RandomUnderSampler
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Modelos preditivos
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier, RandomForestClassifier, VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier

# Avaliação de modelos
from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix

# Interpretabilidade de modelos
from lime import lime_tabular
import shap


# Avaliando os modelos de classificação, calculando
algumas métricas
def avaliar_modelo(y_true, y_pred):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    return accuracy, precision, recall, f1

# Plota a matriz de confusão
def plotar_matriz_confusao(y_true, y_pred, title):

    # Matriz de confusão
    cm = confusion_matrix(y_true, y_pred)

    # Plot da matriz de confusão
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Não Saiu', 'Saiu'],
                yticklabels=['Não Saiu', 'Saiu'])
    plt.xlabel('Predição')
    plt.ylabel('Valor Real')
    plt.title(title)
    plt.show()
```

Código-fonte 9 - Imports e funções de apoio
Fonte: Elaborado pelo autor (2024)

A saída será apenas a confirmação das instalações das bibliotecas de interpretabilidade. Vamos omiti-la e prosseguir para a **leitura dos dados, assim como análise exploratória inicial**:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # CARREGAMENTO DOS DADOS
    data = pd.read_csv('churn_modelling.csv')
    display(data.sample(10))

    # ANÁLISE EXPLORATÓRIA

    #   - Informações gerais do dataset
    display(data.info())

    #   - Análises estatísticas gerais
    display(data.describe())

    #   - Identificação de duplicatas
    display(f"Número de linhas duplicadas:
{data.duplicated().sum()}")
        plt.title(title)
        plt.show()
```

Código-fonte 10 - Leitura dos dados e Análise exploratória
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 10” mostra o resumo da base, contendo 10000 exemplos, sem dados faltantes. O extrato da direita mostra 10 linhas aleatórias, para compreendermos os tipos de dados que a base apresenta.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  object
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts
3846	3847	15737354	Yin	554	France	Female	48	7	0.00	2
602	603	15782569	Stout	687	France	Female	72	9	0.00	1
8250	8251	15688865	Wade	850	France	Female	35	9	0.00	2
8770	8771	15656901	Nnonso	615	France	Male	59	8	0.00	2
587	588	15680998	Nwankwo	725	France	Male	44	5	0.00	1
3584	3585	15664227	Threatt	506	Germany	Male	28	8	53053.76	1
922	923	15753337	Yeates	555	France	Male	51	5	0.00	3
8104	8105	15805413	Chiang	769	France	Female	31	6	117852.26	2
8197	8198	15741180	Eddy	617	France	Male	54	6	102141.90	1
3366	3367	15684010	Tuan	640	Germany	Female	74	2	116800.25	1

(saída parcialmente omitida)

Figura 27 - Saída do código-fonte 10
Fonte: Elaborado pelo autor (2024)

Apesar da base estar completa, sem dados faltantes, **podemos investigar a presença de outliers**, que podem prejudicar a modelagem. Faremos isso por meio de uma inspeção visual via **gráficos boxplot**:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Identificação de outliers via Boxplots:

    #   - Selecionar as variáveis para o boxplot
    features = ['Age', 'Tenure', 'Balance', 'NumOfProducts',
'EstimatedSalary']

    #   - Criar uma figura com 2 linhas e 4 colunas
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    #   - Iterar pelas variáveis e criar o boxplot em cada
subplot
    for i, feature in enumerate(features):
        row = i // 3
        col = i % 3
        sns.boxplot(y=data[feature], ax=axes[row, col])
        axes[row, col].set_title(feature)

    #   - Ajustar o layout da figura
    plt.tight_layout()
    plt.show()
```

Código-fonte 11 - Identificação de Outliers
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 11” mostra que existem outliers nas variáveis AGE e NUMOFPRODUCTS. Como existe apenas 1 outlier para o número de produtos, vamos ignorá-lo e manter a base como está. A variação etária dos clientes pode ser um atributo interessante para ser explorado, portanto, também manteremos essas linhas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

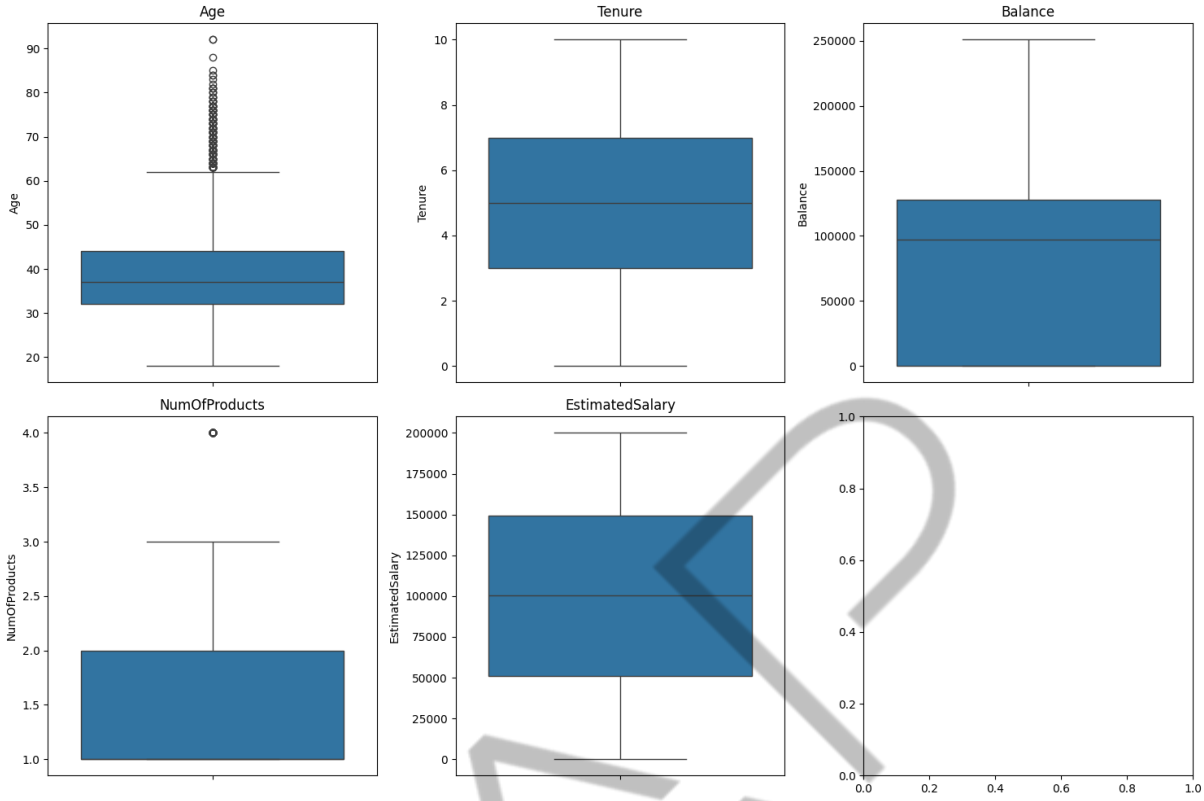


Figura 28 - Saída do código-fonte 11
Fonte: Elaborado pelo autor (2024)

Outra análise interessante é a **correlação das features**, obtida por meio do seguinte comando:

```
# Plot da matriz de correlação com mapa de calor
plt.figure(figsize=(12, 10))
sns.heatmap(data.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm', fmt=".2f")
plt.title('Matriz de Correlação')
plt.show()
```

Código-fonte 12 - Plotando a matriz de correlação
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 12” mostra que as features estão todas fracamente correlacionadas, havendo apenas uma correlação inversa mais significativa entre NUMOFPRODUCTS e BALANCE. Ou seja, menor o saldo, mais produtos o cliente possui (ou vice-versa). Considerando que não há features fortemente correlacionadas entre si, podemos manter todas, a princípio, pois podem ser informativas ao modelo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

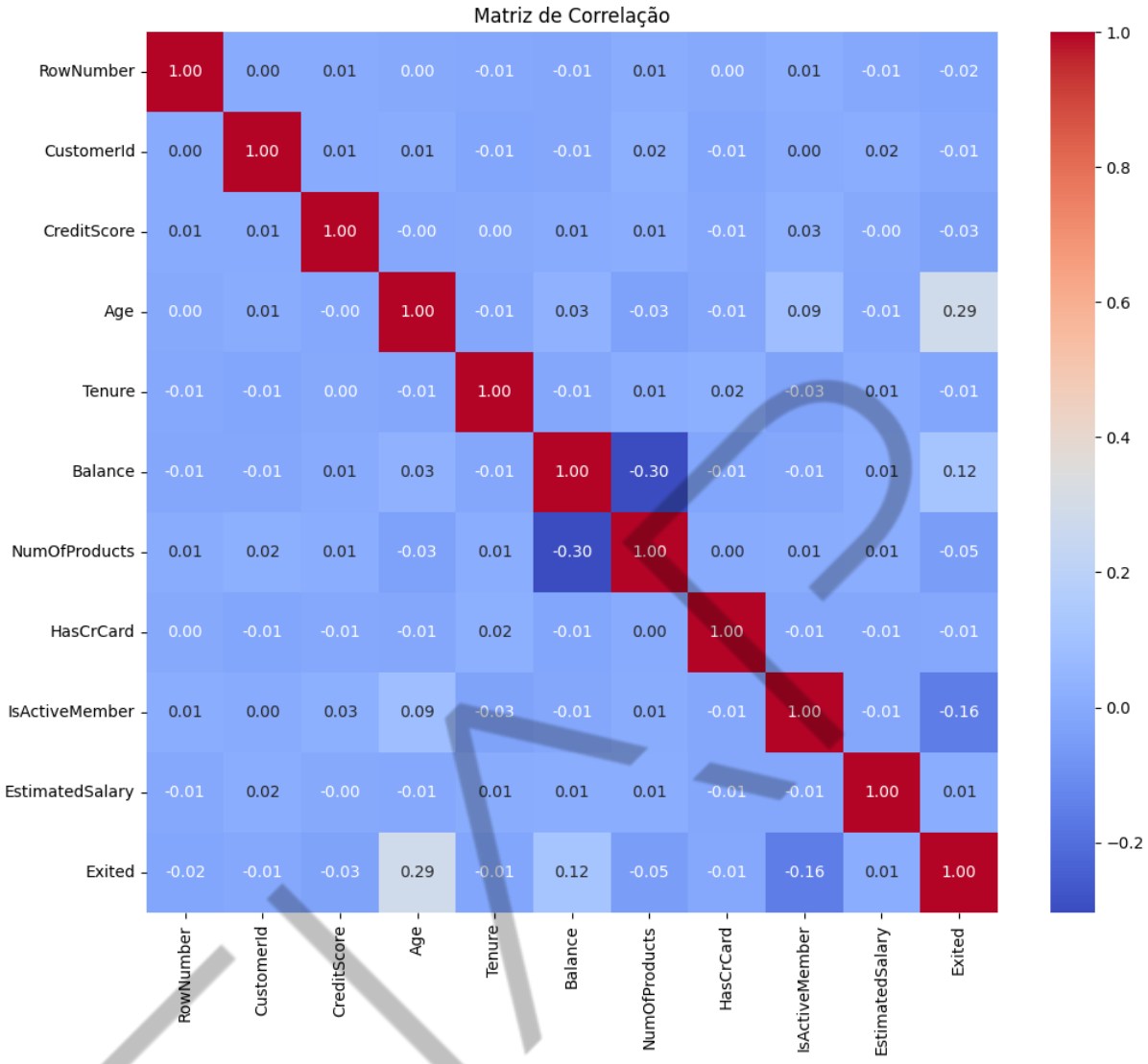


Figura 29 - Saída do código-fonte 12
Fonte: Elaborado pelo autor (2024)

Agora chegou a hora de fazer o **tratamento de dados**, eliminando os duplicados e features que não possuem relação com o problema, como os IDs e o sobrenome do cliente. O código a seguir não possui saída, pois a própria variável `data` está sendo alterada nos dois comandos subsequentes:

```
# Eliminando linhas duplicadas
data.drop_duplicates(inplace=True)

# Eliminando features que não fazem sentido para o problema
data.drop(columns=['RowNumber', 'CustomerId', 'Surname'], inplace=True)
```

Código-fonte 13 - Tratamento dos dados
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos avaliar o **desbalanceamento das classes**:

```
# Plotar o desbalanceamento da variável "Exited"
plt.figure(figsize=(6, 4))
sns.countplot(x='Exited', data=data)
plt.title('Distribuição da variável Exited')
plt.xlabel('Exited')
plt.ylabel('Contagem')
plt.show()
```

Código-fonte 14 - Plotando a distribuição das classes
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 14” mostra que as classes não estão balanceadas, ou seja, há muitos exemplos de pessoas que não abandonaram os serviços bancários (~8k) e poucos que abandonaram (~2k).

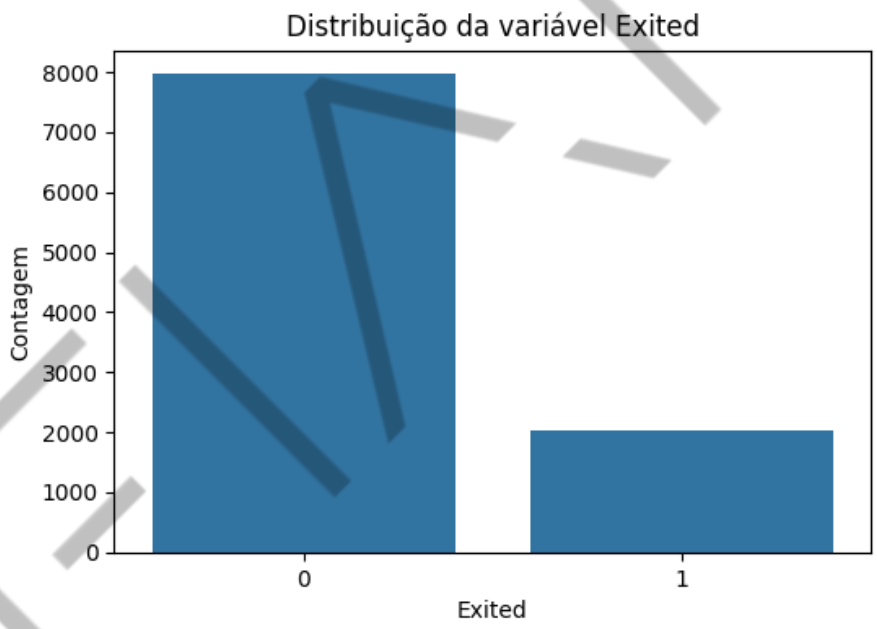


Figura 30 - Saída do código-fonte 14
Fonte: Elaborado pelo autor (2024)

Portanto, realizar alguma ação para **mitigar o desbalanceamento** pode ser relevante. Aqui não queremos eliminar o desbalanceamento por completo, apenas atenuá-lo. Podemos manter todos os exemplos de “*churn*” e eliminar 40% dos dados de “não *churn*” da seguinte forma:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Separar as features (X) e o target (y)
X = data.drop('Exited', axis=1)
y = data['Exited']

# Criar o objeto RandomUnderSampler
rus = RandomUnderSampler(sampling_strategy={0:
int(y.value_counts()[0] * 0.4)}, random_state=42)

# Aplicar o undersampling aos dados
X_resampled, y_resampled = rus.fit_resample(X, y)

# Criar um novo DataFrame com os dados reamostrados
data_resampled = pd.concat([X_resampled, y_resampled],
axis=1)

# Verificar a contagem de cada classe após o undersampling
plt.figure(figsize=(6, 4))
sns.countplot(x='Exited', data=data_resampled)
plt.title('Distribuição da variável Exited')
plt.xlabel('Exited')
plt.ylabel('Contagem')
plt.show()
```

Código-fonte 15 - Atenuando o desbalanceamento das classes
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 15” mostra que as classes mais bem balanceadas. Temos agora ~3k exemplos de não abandono contra os ~2k exemplos de abandono.

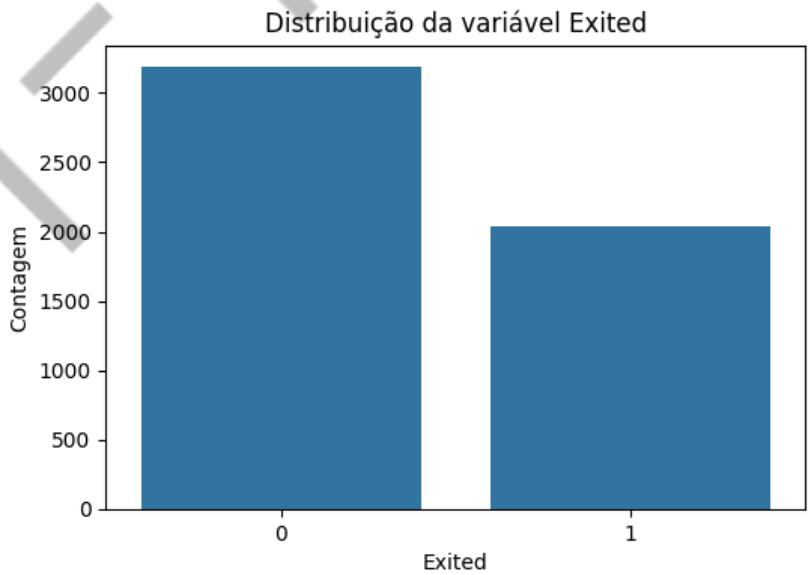


Figura 31 - Saída do código-fonte 15
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para complementar esta etapa inicial, **vamos realizar a engenharia de features**, aplicando OneHotEncoder, separando o TREINO e o TESTE, bem como padronizando as variáveis resultantes. O código a seguir não possui saídas.

```
# Convertendo variáveis categóricas para numéricas (se necessário)
X_resampled = pd.get_dummies(data_resampled, drop_first=True)

# Separando os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X_resampled.drop('Exited', axis=1),
                                                    X_resampled['Exited'],
                                                    test_size=0.2,
                                                    random_state=42)

# Padronizando as features
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

Código-fonte 16 - Engenharia de Features
Fonte: Elaborado pelo autor (2024)

Para viabilizar a comparação entre modelos preditivos, vamos começar com um **modelo baseline, baseado na classificação linear de uma Regressão Logística**. O desempenho será avaliado por meio de múltiplas métricas e da matriz de confusão, ilustrada na figura “Saída do código-fonte 17”.

```
# Modelo baseline (classificação linear)
baseline_model = LogisticRegression()
baseline_model.fit(X_train_scaled, y_train)
y_pred_baseline = baseline_model.predict(X_test_scaled)
accuracy_baseline, precision_baseline, recall_baseline, f1_baseline = avaliar_modelo(y_test, y_pred_baseline)
print(f"BASELINE: Acurácia: {accuracy_baseline:.4f}, Precisão: {precision_baseline:.4f}, Recall: {recall_baseline:.4f}, F1-Score: {f1_baseline:.4f}")

plotar_matriz_confusao(y_test, y_pred_baseline, "Matriz de Confusão - Baseline")
```

Código-fonte 17 - Gerando o modelo baseline
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

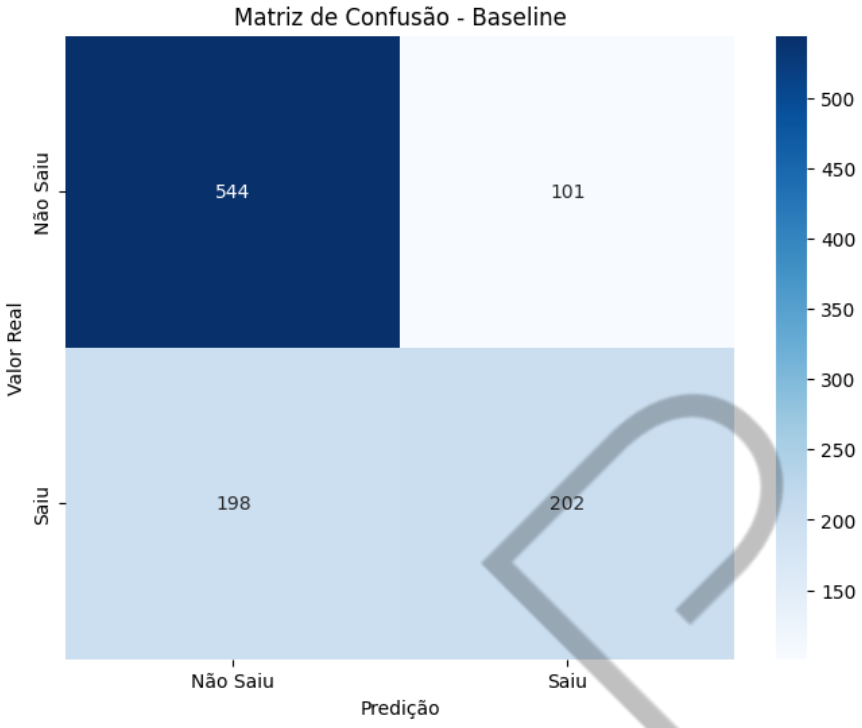


Figura 32 - Saída do código-fonte 17
Fonte: Elaborado pelo autor (2024)

7.3 Ensembles

Uma vez que temos um modelo baseline, podemos tentar criar modelos melhores por diversas estratégias: ou aplicando algoritmos mais sofisticados ou lançando mão de técnicas de ensemble.

Na sequência, **vamos criar 1 modelo baseado em XGBoost e 4 ensembles distintos**, empregando as técnicas de *bagging*, *boosting*, *voting* ou *stacking*. Nossos modelos de base serão árvores de decisão, florestas aleatórias, KNN e SVM. Outros modelos poderiam ser explorados.

```
    # Bagging - Combinação de 100 árvores independentes
(similar à Floresta)
    bagging_model                                          =
BaggingClassifier(estimator=DecisionTreeClassifier(),
                                      n_estimators=100,
                                      random_state=42)
    bagging_model.fit(X_train_scaled, y_train)
    y_pred_bagging = bagging_model.predict(X_test_scaled)

    # Boosting
    boosting_model = AdaBoostClassifier(n_estimators=100,
                                        random_state=42)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    boosting_model.fit(X_train_scaled, y_train)
    y_pred_boosting = boosting_model.predict(X_test_scaled)

    # Definindo modelos base para ensembles para VOTING e
STACKING
    base_models = [
          ('dt', DecisionTreeClassifier(random_state=42)),
             ('rf',  RandomForestClassifier(n_estimators=100,
random_state=42)),
          ('knn', KNeighborsClassifier()),
          ('svm', SVC())
    ]

    # Voting - Votação entre os modelos de base
    voting_model = VotingClassifier(estimators=base_models,
                                    voting='hard')
    voting_model.fit(X_train_scaled, y_train)
    y_pred_voting = voting_model.predict(X_test_scaled)

    # Stacking
    stacking_model                                          =
StackingClassifier(estimators=base_models,
                                        final_estimator=Logi
sticRegression())
    stacking_model.fit(X_train_scaled, y_train)
    y_pred_stacking = stacking_model.predict(X_test_scaled)

    # XGBoost
    xgb_model = XGBClassifier(random_state=42)
    xgb_model.fit(X_train_scaled, y_train)
    y_pred_xgb = xgb_model.predict(X_test_scaled)

    # Avaliação dos modelos
    accuracy_bagging,   precision_bagging,   recall_bagging,
f1_bagging = avaliar_modelo(y_test, y_pred_bagging)
    accuracy_boosting,  precision_boosting,  recall_boosting,
f1_boosting = avaliar_modelo(y_test, y_pred_boosting)
    accuracy_voting,    precision_voting,    recall_voting,
f1_voting = avaliar_modelo(y_test, y_pred_voting)
    accuracy_stacking,  precision_stacking,  recall_stacking,
f1_stacking = avaliar_modelo(y_test, y_pred_stacking)
    accuracy_xgb,   precision_xgb,   recall_xgb,   f1_xgb   =
avaliar_modelo(y_test, y_pred_xgb)

    print(f"Bagging:\tAcurácia: {accuracy_bagging:.4f}, \
            Precisão: {precision_bagging:.4f}, \
            Recall: {recall_bagging:.4f}, \
            F1-Score: {f1_bagging:.4f}")
    print(f"Boosting:\tAcurácia: {accuracy_boosting:.4f}, \
            Precisão: {precision_boosting:.4f}, \
            Recall: {recall_boosting:.4f}, \
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                F1-Score: {f1_boosting:.4f}")
    print(f"Voting:\t\tAcurácia: {accuracy_voting:.4f}, \
                Precisão: {precision_voting:.4f}, \
                Recall: {recall_voting:.4f}, \
                F1-Score: {f1_voting:.4f}")
    print(f"Stacking:\tAcurácia: {accuracy_stacking:.4f}, \
                Precisão: {precision_stacking:.4f}, \
                Recall: {recall_stacking:.4f}, \
                F1-Score: {f1_stacking:.4f}")
    print(f"XGBoost:\tAcurácia: {accuracy_xgb:.4f}, \
                Precisão: {precision_xgb:.4f}, \
                Recall: {recall_xgb:.4f}, \
                F1-Score: {f1_xgb:.4f}")
```

Código-fonte 18 - Gerando modelos por diferentes estratégias de ensemble
Fonte: Elaborado pelo autor (2024)

Após as definições e treinamentos, todos os modelos são avaliados pelas mesmas métricas do modelo baseline, para viabilizar comparações (figura “Saída do código-fonte 18”).

```
Bagging:        Acurácia: 0.8077,        Precisão: 0.7741,        Recall: 0.7025,        F1-Score: 0.7366
Boosting:       Acurácia: 0.8019,        Precisão: 0.7718,        Recall: 0.6850,        F1-Score: 0.7258
Voting:         Acurácia: 0.8134,        Precisão: 0.8571,        Recall: 0.6150,        F1-Score: 0.7162
Stacking:       Acurácia: 0.8153,        Precisão: 0.7983,        Recall: 0.6925,        F1-Score: 0.7416
XGBoost:        Acurácia: 0.7971,        Precisão: 0.7527,        Recall: 0.7000,        F1-Score: 0.7254
```

Figura 33 - Saída do código-fonte 18
Fonte: Elaborado pelo autor (2024)

Também podemos traçar comparações mais aprimoradas por meio **de matrizes de confusão**, obtidas com os códigos a seguir, concentradas na figura “Saída do código-fonte 19”:

```
    # Plota as matrizes de confusão para cada modelo
    plotar_matriz_confusao(y_test, y_pred_baseline, "Matriz de Confusão - Baseline")
    plotar_matriz_confusao(y_test, y_pred_bagging, "Matriz de Confusão - Bagging")
    plotar_matriz_confusao(y_test, y_pred_boosting, "Matriz de Confusão - Boosting")
    plotar_matriz_confusao(y_test, y_pred_voting, "Matriz de Confusão - Voting")
    plotar_matriz_confusao(y_test, y_pred_stacking, "Matriz de Confusão - Stacking")
    plotar_matriz_confusao(y_test, y_pred_xgb, "Matriz de Confusão - XGBoost")
```

Código-fonte 19 - Plotando matrizes de confusão
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

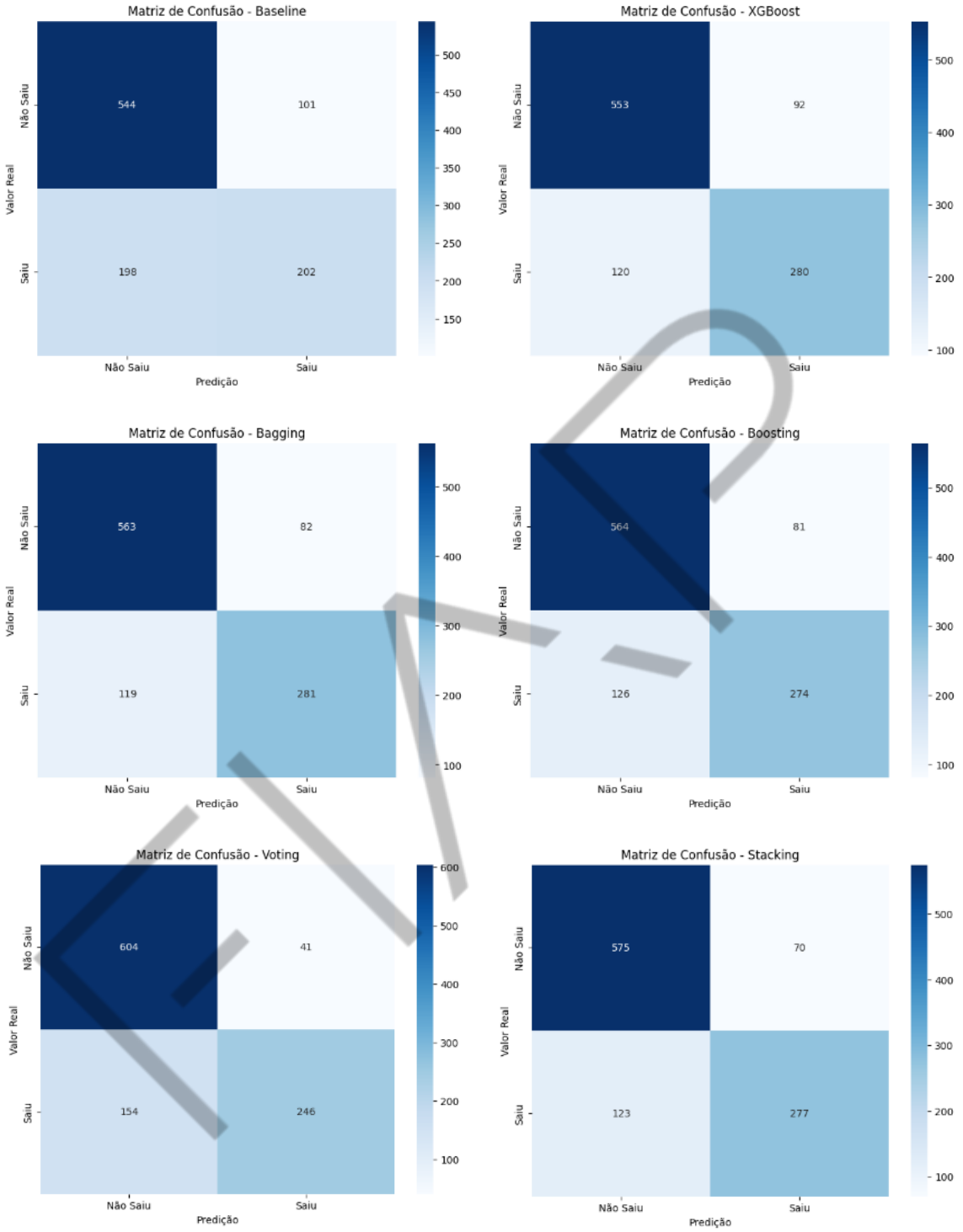


Figura 34 - Saída do código-fonte 19
Fonte: Elaborado pelo autor (2024)

Pelas métricas da figura “Saída do código-fonte 17” e das matrizes de confusão da figura “Saída do código-fonte 18”, notamos que o melhor modelo é o ensemble baseado em **Stacking**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7.4 Interpretabilidade

Para concluir, podemos avançar a análise por meio da **interpretação dos resultados**. Para começar, vamos passar o 100º exemplo de teste para o **LIME**, obtendo as justificativas da predição pelo método de *stacking*. Da seguinte forma:

```
# LIME
explainer = lime_tabular.LimeTabularExplainer(
    X_train_scaled,
    feature_names=X_test.columns.tolist(),
    class_names=['Não Saiu', 'Saiu'],
    mode='classification'
)

# Escolha um índice da amostra de teste para explicar
i = 100
exp = explainer.explain_instance(
    X_test_scaled[i],
    stacking_model.predict_proba,
    num_features=10
)
exp.show_in_notebook(show_table=True)
```

Código-fonte 20 - Explicando um exemplo de teste via LIME
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 20” mostra a explicabilidade do modelo para uma instância em questão. Ela foi classificada como “não *churn*”, sendo que o modelo estava 79% seguro desta resposta, porém pendendo 21% para que o usuário tivesse dados *churn*. O que mais contribuiu para não ser *churn* foi seu valor de “é membro ativo”, “número de produtos” e “geografia == Espanha”. Outras features contribuíram na direção oposta. Os números da tabela e do gráfico podem não fazer sentido, mas lembre-se que o modelo foi feito sobre um dataset padronizado com StandardScale().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

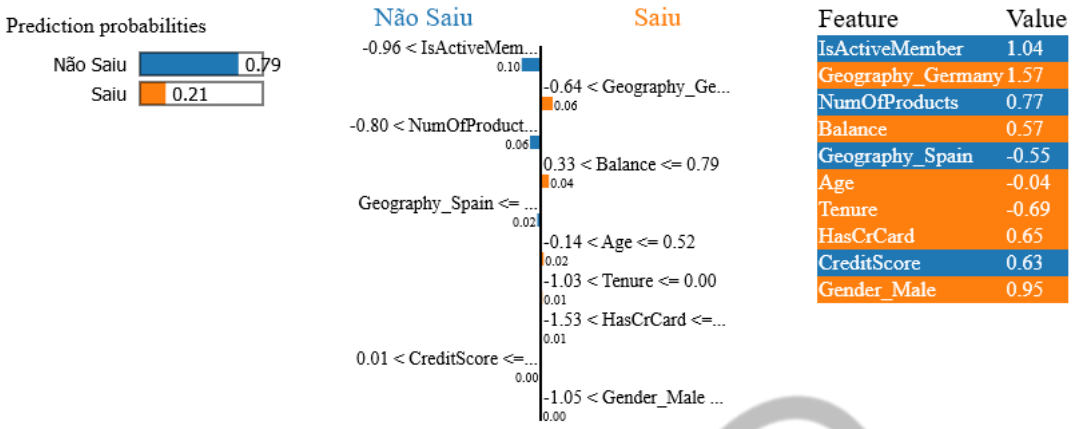


Figura 35 - Saída do código-fonte 20
Fonte: Elaborado pelo autor (2024)

Também podemos construir uma análise de features pelo **método do SHAP**. Aqui vamos avaliar como o XGBoost classificaria os exemplos de acordo com suas features.

```
     # SHAP
     explainer        =        shap.Explainer(xgb_model.predict,
X_test_scaled)
     shap_values = explainer(X_test_scaled)

     # Resumo das importâncias das features
     shap.summary_plot(shap_values,            X_test_scaled,
feature_names=X_test.columns.tolist())
```

Código-fonte 21 - Explicabilidade geral do modelo via SHAP Values
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 21” mostra que as features mais importantes para os modelos são AGE, NUMOFPRODUCTS e ISACTIVEMEMBER. Iremos usar como exemplo a feature NUMOFPRODUCTS, ao utilizá-la, é esperado que quanto mais produtos um determinado cliente adquiriu, mais ele é fidelizado ao banco. Porém, o diagrama mostra exatamente o oposto. Quanto maior (*high*) o valor desta variável, mais ela irá puxar o score do método para evasão (SHAP Value positivo → para a direita do gráfico). Ao inspecionar os dados encontramos também esta relação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

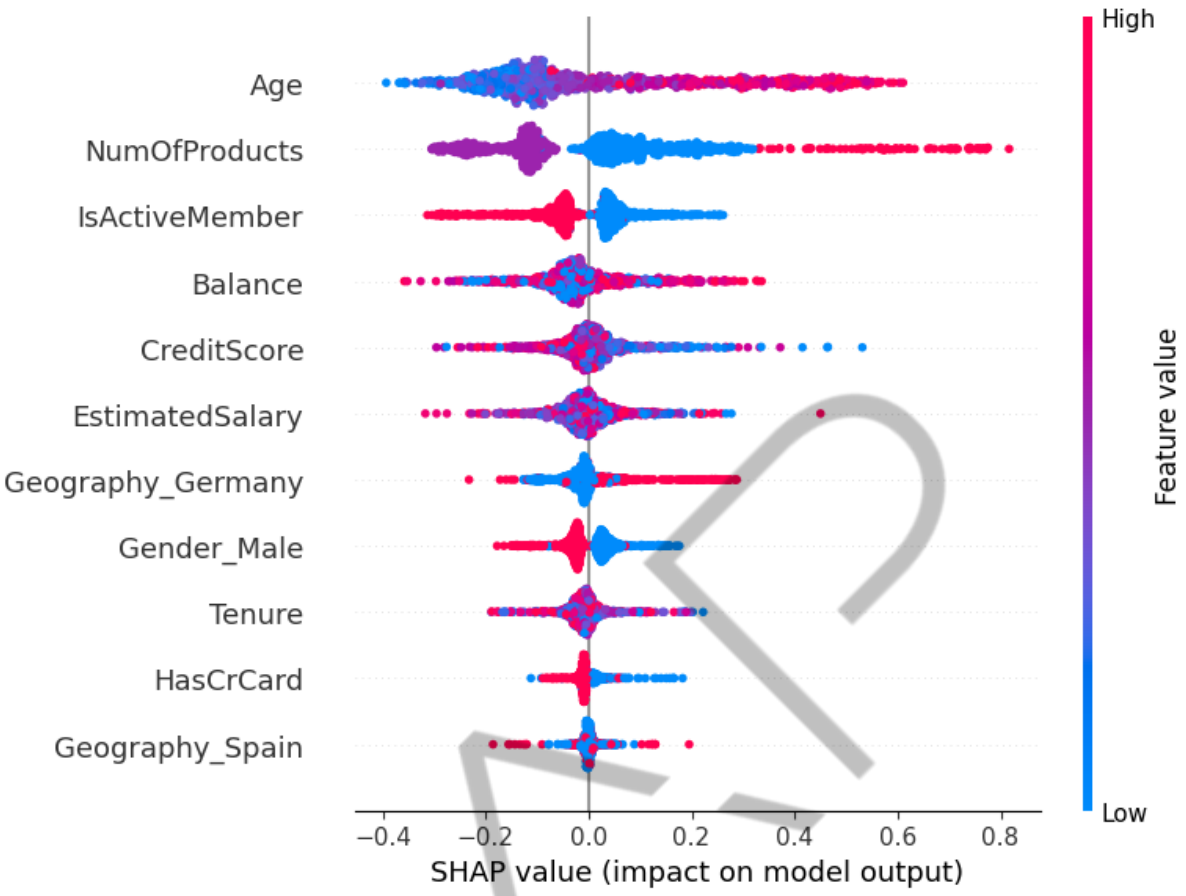


Figura 36 - Saída do código-fonte 21
Fonte: Elaborado pelo autor (2024)

E então? Quais outras conclusões você tiraria a partir destes gráficos?

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Neste capítulo, exploramos **tópicos avançados de Machine Learning**, incluindo formas de **preparação de dados** e técnicas para lidar com problemas específicos, como **desbalanceamento** e **escolha de features**. Abordamos métodos preditivos de **ensemble**, como *bagging*, *boosting*, *voting* e *stacking*, que fortalecem o desempenho dos modelos ao combinar diferentes abordagens para reduzir viés e variância, resultando em previsões mais robustas e precisas. Além disso, discutimos a importância da **interpretabilidade**, destacando como ela é fundamental para garantir a confiança e a transparência nas decisões automatizadas. Também introduzimos o **PyCaret**, que facilita o trabalho de Machine Learning, e várias ferramentas, como **LIME** e **SHAP**, para avaliar a interpretabilidade dos modelos.

Finalmente, conhecemos o aprendizado por reforço e o algoritmo Q-Learning, que ensina agentes a aprender com a própria experiência e se adaptar a ambientes dinâmicos. O aprendizado por reforço exemplifica a capacidade dos modelos de Machine Learning de aprender com feedbacks e otimizar suas ações, sendo aplicável a uma variedade de contextos, desde jogos até decisões em tempo real em robótica e automação.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BERTOLINI, Massimo *et al*. **Machine Learning for industrial applications:** A comprehensive literature review. Expert Systems with Applications, 2021, 175: 114820.

BROWNLEE, Jason. **Data preparation for machine learning:** data cleaning, feature selection, and data transforms in Python. Machine Learning Mastery, 2020.

CANDIDO, Gustavo. Ensemble de Modelos. **Medium**, 20 de novembro de 2023. Disponível em: <https://medium.com/@lg0702/ensemble-de-modelos-7f17999761be>. Acesso em: 28 nov. 2024.

CHAWLA, Avi. An Animated Guide to Bagging and Boosting in Machine Learning. **Daily Dose of Data Science**, 18 de outubro de 2023. Disponível em: <https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and>. Acesso em: 28 nov. 2024.

DARA, Maher. Balance Exploitation & Exploration within Your Organization with TOGAF. **Bizzdesign**, 31 de março de 2015. Disponível em: <https://bizzdesign.com/blog/balance-exploitation-exploration-within-your-organization-with-togaf/>. Acesso em: 28 nov. 2024.

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn**, Keras, and TensorFlow. O'Reilly Media, Inc., 2022.

GUIDA, Sergio. Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International. **ResearchGate**, janeiro de 2022. Disponível em: <https://www.researchgate.net/figure/The-main-deep-learning-models-in-a-graph-correlating-accuracy-and-interpretability_fig3_365781922>. Acesso em: 28 nov. 2024.

HAMAD, Mothana. Gradient boosting. **YouTube**, 28 de agosto de 2018. Disponível em: <https://www.youtube.com/watch?app=desktop&v=6HCO9WhtV4o>. Acesso em: 28 nov. 2024.

KARUTURI, D., PENDYALA, K., SUDHA, N. **Effective Heart Disease Prediction Using Hybrid Ensemble Learning Model**. In: Sisodia, D. S., Garg, L., Pachori, R. B., Tanveer, M. (eds) Machine Intelligence Techniques for Data Analysis and Signal Processing. Lecture Notes in Electrical Engineering, 2023, vol. 997. Springer, Singapore.

KARUTURI, D.; PENDYALA, K.; SUDHA, N. **Effective Heart Disease Prediction Using Hybrid Ensemble Learning Model**. In: Machine Intelligence Techniques for Data Analysis and Signal Processing: Proceedings of the 4th International Conference MISP 2022, Vol. 1. Singapore: Springer Nature Singapore, 2023. p. 117-125.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

KOZYRKOV, Cassie. What is bias? **Medium**, 24 de janeiro de 2024. Disponível em: <https://towardsdatascience.com/what-is-ai-bias-6606a3bcb814>. Acesso em: 28 nov. 2024.

KUNAPULI, Gautam. **Ensemble methods for machine learning**. Simon and Schuster, 2023.

MACHINE Learning 1 Limited. 10 Innovative Approaches to Machine Learning. **LinkedIn**, 13 de junho de 2024. Disponível em: <https://www.linkedin.com/pulse/10-innovative-approaches-machine-learning-machine-learning-1-jukgf/>. Acesso em: 28 nov. 2024.

MIRSHERAKI, Saeed. Explainable AI: A Guide to Interpreting Classification Models with Lime. **O'Fallon Labs**, 09 de junho de 2023. Disponível em: <https://saeedmirshekari.com/blog/explaining-wine-quality-predictions-with-lime-a-guide-to-interpreting-classification-models/>. Acesso em: 28 nov. 2024.

OTTENHOFF, M.; RAMOS, L. Creative Commons Attribution-NonCommercial 4.0 International. **ResearchGate**, julho de 2021. Disponível em: <https://www.researchgate.net/figure/SHAP-values-of-XGB-trained-on-all-features-To-prevent-readability-issues-only-the-top_fig4_353365970>. Acesso em: 28 nov. 2024.

REFORÇO positivo adestramento e metodologia. **Adestramento de cães**, 16 de janeiro de 2024. Disponível em: <https://www.adestramentoriopreto.com.br/reforco-positivo-adestramento-e-metodologia/>. Acesso em: 28 nov. 2024.

SANTOS, Einar Cesar. 3: Modelo de aprendizado por reforço. **ResearchGate**, junho de 2019. Disponível em: <https://www.researchgate.net/figure/Figura-33-Modelo-de-aprendizado-por-reforco-Extraido-traduzido-e-adaptado-de-89_fig7_335311074>. Acesso em: 28 nov. 2024.

SOLANKI, Sunny. How to Use eli5 to Interpret ML Models and their Predictions? **Coderzcolumn**, 09 de agosto de 2022. Disponível em: <https://coderzcolumn.com/tutorials/machine-learning/how-to-use-eli5-to-understand-sklearn-models-their-performance-and-their-predictions>. Acesso em: 28 nov. 2024.

SUTTON, Richard S. **Reinforcement learning**: An introduction. A Bradford Book, 2018.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	**Explicação**
AdaBoost	Adaptive Boosting.
ELI5	Explain Like I'm 5 years old.
LIME	Local Interpretable Model-agnostic Explanations.
NLP	Natural Language Processing.
RL	Reinforcement Learning.
SHAP	SHapley Additive exPlanations.
SMOTE	Synthetic Minority Over-sampling Technique.
SVM	Máquinas de Vetores de Suporte.
XGBoost	Extreme Gradient Boosting.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com